



المدرسة الدولية الخاصة للتقنيات
POLYTECH INTL

École Polytechnique Privée Internationale de Tunis

Cycle Ingénieur – Génie Informatique

Spécialité : Systèmes d'Information et Génie Logiciel

Rapport de Stage de Fin d'Études

Implémentation d'un framework de test intelligent avec
auto-réparation basé sur l'IA pour une application web réelle

Réalisé par

Asma Hammami

Stage effectué au sein de

MOONSIDE CONSULTING & SERVICES

Encadré par

Encadrant pédagogique : M. Ghazouani Haythem

Encadrant professionnel : M. Faker Sofiene

« Les grandes réalisations sont toujours accomplies avec le soutien des êtres chers. »

Je dédie ce travail

*À mes très chers parents,
qui n'ont jamais cessé de me soutenir, de m'encourager et de croire en moi
tout au long de mon parcours.
Qu'ils trouvent ici l'expression de ma profonde gratitude et de mon amour
infini.*

*À mes frères, à mes grands-parents et à toute ma famille,
pour leur présence constante, leur affection sincère et leur soutien
inconditionnel.*

*À toutes les personnes qui m'ont aidée, de près ou de loin,
et qui ont contribué à la réalisation de ce travail.*

*À mes chers amis,
pour leurs encouragements et leur énergie positive.*

Merci du fond du cœur.

Asma

Remerciements

Je tiens, avant de présenter mon travail, à exprimer ma profonde gratitude et ma reconnaissance envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Je souhaite adresser mes sincères remerciements à l'entreprise **Moonside Consulting & Services** pour m'avoir accueillie au sein de son équipe et m'avoir offert un environnement de travail enrichissant, stimulant et propice à l'apprentissage.

Je tiens à remercier particulièrement mon encadrant professionnel, **M. Faker Sofiene**, pour son accompagnement, sa disponibilité, ses conseils précieux ainsi que pour la confiance qu'il m'a accordée tout au long de ce stage.

Mes remerciements s'adressent également à mon encadrant pédagogique, **M. Ghazouani Haythem**, pour son suivi, ses orientations pertinentes et ses encouragements continus qui ont grandement contribué à l'aboutissement de ce travail.

Je tiens aussi à exprimer ma reconnaissance envers l'ensemble du corps enseignant de l'**École Polytechnique Privée Internationale de Tunis** pour la qualité de la formation qui m'a été dispensée, ainsi que pour leur engagement et leur soutien tout au long de mon parcours académique.

Enfin, je remercie chaleureusement ma famille et mes amis pour leur soutien moral, leur patience et leurs encouragements constants, qui ont été une source essentielle de motivation durant la réalisation de ce projet.

Je tiens également à remercier les membres du jury pour l'intérêt qu'ils porteront à ce travail et pour le temps qu'ils consacreront à son évaluation.

Asma Hammami

Résumé

Le présent rapport constitue une synthèse de mon projet de fin d'études réalisé au sein de **Moonside Consulting & Services**, dans le cadre de l'obtention du diplôme d'ingénieur en génie informatique, spécialité systèmes d'information et génie logiciel.

Ce projet porte sur la conception et l'implémentation d'un framework intelligent de tests automatisés capable de s'auto-réparer face aux changements dynamiques des applications web. L'objectif principal est d'améliorer la robustesse et la maintenabilité des tests automatisés en intégrant des techniques d'intelligence artificielle.

La solution proposée permet de détecter les défaillances des tests causées par des modifications de l'interface utilisateur, et de générer automatiquement des alternatives pertinentes aux sélecteurs défaillants. Elle vise ainsi à réduire les coûts de maintenance et à garantir la continuité des campagnes de test.

Ce travail s'appuie sur une étude comparative entre les approches classiques d'automatisation des tests et les solutions intelligentes basées sur l'intelligence artificielle. Il intègre également des outils modernes de test ainsi qu'une démarche d'intégration continue afin d'assurer une meilleure qualité logicielle.

Le projet a été mené en adoptant une méthodologie agile, favorisant la collaboration, l'itération et l'adaptation aux besoins évolutifs.

Mots-clés : Tests automatisés, Auto-réparation, Intelligence artificielle, Applications web, Qualité logicielle, CI/CD.

Abstract

This report presents a summary of my final year project carried out at **Moonside Consulting & Services**, as part of the requirements for obtaining an engineering degree in Software Engineering and Information Systems.

The project focuses on the design and implementation of an intelligent automated testing framework capable of self-healing in response to dynamic changes in web applications. The main objective is to enhance the robustness and maintainability of automated tests by leveraging artificial intelligence techniques.

The proposed solution detects test failures caused by user interface changes and automatically generates alternative selectors to recover broken tests. This approach significantly reduces maintenance efforts and ensures the continuity of testing processes.

This work includes a comparative study between traditional test automation approaches and AI-based intelligent solutions. It also integrates modern testing tools along with continuous integration practices to improve software quality.

The project was conducted using an agile methodology, promoting adaptability, collaboration, and iterative development.

Keywords : Automated Testing, Self-healing, Artificial Intelligence, Web Applications, Software Quality, CI/CD.

Table des matières

Liste des sigles et acronymes	10
Introduction	11
1 Contexte général du projet	12
1.1 Introduction	12
1.2 Présentation de l'organisme d'accueil	12
1.2.1 Carte d'identité	12
1.2.2 Métiers de Moonside Consulting & Services	13
1.2.3 Organigramme	14
1.2.4 Valeurs fondamentales	15
1.3 Contexte du projet	16
1.3.1 Cadre du projet	16
1.3.2 Problématique	16
1.3.3 Objectifs et périmètre	18
1.3.4 Solution proposée	18
1.4 Planification et conduite du projet	20
1.4.1 Processus de développement	20
1.4.2 Planning du projet	20
1.5 Conclusion	22
2 État de l'art et solution proposée	23
2.1 Introduction	23
2.2 État de l'art — Outils et approches de <i>self-healing</i>	23
2.2.1 Panorama général	23
2.2.2 Healenium	24
2.2.3 Testim	25
2.2.4 mabl	25
2.2.5 Applitools	26

2.2.6	Cypress et Playwright natifs	27
2.2.7	Travaux académiques	27
2.2.8	Tableau comparatif synthétique	28
2.2.9	Comparaison expérimentale des stratégies de résilience	28
2.2.10	Lacunes identifiées et positionnement de la solution	30
2.3	Motivation et principes de conception	30
2.3.1	Insuffisances des approches conventionnelles	30
2.3.2	Principes directeurs de la conception	31
2.4	Analyse des besoins	32
2.4.1	Contexte applicatif et scénarios de référence	32
2.4.2	Identification des acteurs	32
2.4.3	Besoins fonctionnels	33
2.4.4	Besoins non fonctionnels	35
2.4.5	Diagramme des cas d'utilisation	36
2.4.6	Contraintes et hypothèses de conception	36
2.5	Architecture générale du système	37
2.5.1	Vue d'ensemble des composants	37
2.5.2	Mécanisme de persistance de la référence	38
2.5.3	Flux de traitement global	38
2.6	Pipeline de décision en quatre étapes	40
2.6.1	Étape 1 — Analyse sémantique par Sentence-BERT	40
2.6.2	Étape 2 — Analyse contextuelle par attributs ARIA	41
2.6.3	Étape 3 — Analyse textuelle par TF-IDF	41
2.6.4	Étape 4 — Analyse visuelle par SSIM	41
2.7	Génération des locators de remplacement	42
2.8	Interface d'intégration : l'API REST	43
2.8.1	Conception	43
2.8.2	Gestion des requêtes et déploiement	45
2.9	Choix technologiques	45
2.9.1	Justification du choix du modèle sémantique	46
2.10	Conclusion	48
3	Business Understanding : cadrage métier du self-healing ML	49
3.1	Introduction	49
3.2	Problématique métier	49
3.3	Objectifs du projet	50

3.4	Critères de succès	50
3.5	Architecture ML visée	51
3.6	Contraintes et hypothèses	52
3.7	Planification CRISP-DM	52
3.8	Conclusion	52
4	Data Understanding : compréhension des données du projet	53
4.1	Introduction	53
4.2	Sources de données	53
4.2.1	Page web cible	54
4.2.2	DOM extrait par Playwright	54
4.2.3	Scénarios d'évaluation	55
4.3	Analyse du vocabulaire UI	55
4.4	Qualité et limites des données	56
4.5	Conclusion	56
5	Data Preparation : préparation des données pour le pipeline ML	57
5.1	Introduction	57
5.2	Architecture de préparation	58
5.3	Parsing du sélecteur	58
5.4	Feature engineering multi-modal	59
5.5	Détection des sélecteurs manifestement invalides	60
5.6	Extraction et filtrage du DOM	61
5.7	Construction du texte de référence	62
5.8	Enrichissement lexical	63
5.9	Préparation des représentations sémantiques et visuelles	64
5.10	Conclusion	65
6	Modeling : conception du moteur ML de self-healing	66
6.1	Introduction	66
6.2	Vue d'ensemble du pipeline de scoring	66
6.3	Modèle de décision ML	67
6.4	Fonction de fusion des scores	68
6.5	Stage 1 : similarité sémantique	68
6.6	Stage 2 : scoring contextuel	68
6.7	Stage 3 : similarité textuelle TF-IDF	68
6.8	Stage 4 : similarité visuelle SSIM	68

6.9	Stage 5 : vision IA par CNN	69
6.10	Génération des locators	69
6.11	Conclusion	69
7	Evaluation : validation du pipeline ML de self-healing	70
7.1	Introduction	70
7.2	Protocole d'évaluation	70
7.3	Matrice de confusion	71
7.4	Résultats et évolution des métriques	71
7.5	Analyse des causes racines	72
7.6	Performances temporelles	72
7.7	Conclusion	73
8	Deployment : intégration et mise à disposition du système ML	74
8.1	Introduction	74
8.2	Architecture de déploiement	74
8.3	Séquence d'intégration	75
8.4	Composant moteur : notebook Colab	75
8.5	API Flask et endpoints	76
8.6	Client Playwright	76
8.7	Dashboard Angular	76
8.8	Sécurité et bonnes pratiques	77
8.9	Conclusion	77
	Conclusion générale	78

Table des figures

1.1	Logo de Moonside Consulting & Services	12
1.2	Vue d'ensemble des services de Moonside Consulting & Services	14
1.3	Organigramme simplifié de Moonside Consulting & Services	15
1.4	Cycle de vie d'un test automatisé et point de fragilité des sélecteurs	16
1.5	Impact d'une modification du DOM sur la validité d'un sélecteur	17
1.6	Architecture du système de <i>self-healing</i> proposé	19
1.7	Les six phases de la méthodologie CRISP-DM	20
1.8	Diagramme de Gantt — planning prévisionnel du stage (02/02–31/07/2026) et clôture effective le 04/07/2026	21
2.1	Cartographie des outils de <i>self-healing</i> — positionnement selon l'approche technique	24
2.2	Logo de Healenium	24
2.3	Logo de Testim	25
2.4	Logo de mabl	25
2.5	Logo de Applitools	26
2.6	Logos de Cypress et Playwright	27
2.7	Diagramme des cas d'utilisation — acteurs et interactions principales	36
2.8	Architecture générale du système de <i>self-healing</i> — quatre modules princi- paux	37
2.9	Diagramme de séquence — flux de traitement complet du moteur de <i>self- healing</i>	39
2.10	Entonnoir de réduction progressive des candidats à travers les quatre étapes	40
2.11	Calcul du score composite — pondération des couches et évolution des scores des candidats	42
2.12	Hierarchie des locators générés — classement par stabilité avec exemples de code	43
2.13	Interface API REST — points de terminaison et flux d'intégration avec ngrok	44
2.14	Arbitrage entre qualité de localisation et latence des approches candidates	47
2.15	Matrice de décision multicritère pour le choix du modèle sémantique	48

3.1	Vision globale de l'architecture ML du moteur ARCANE	51
3.2	Planification du projet ARCANE selon les six phases CRISP-DM	52
4.1	Sources de données exploitées par le pipeline de self-healing	54
4.2	Répartition des scénarios utilisés pour tester le moteur ML	55
5.1	Rôle de la préparation des données dans la chaîne de décision ARCANE . .	58
5.2	Pipeline de préparation des données avant modélisation	58
5.3	Décomposition d'un sélecteur cassé en informations exploitables	59
5.4	Passage d'une chaîne CSS brute vers des signaux sémantiques, lexicaux et DOM	59
5.5	Construction des caractéristiques utilisées par les modèles ML	60
5.6	Filtre précoce des sélecteurs manifestement invalides	61
5.7	Garde-fou expérimental contre les faux positifs de healing	61
5.8	Extraction DOM et filtrage des candidats non pertinents	62
5.9	Construction d'une représentation textuelle unifiée pour chaque candidat DOM	62
5.10	Construction progressive d'une baseline textuelle à partir du sélecteur cassé	63
5.11	Enrichissement lexical à partir du vocabulaire fonctionnel des interfaces web	64
5.12	Préparation conjointe des représentations sémantiques et visuelles	64
5.13	La préparation des données comme couche de médiation entre Playwright et le scoring ML	65
6.1	Entonnoir de scoring multi-étages du moteur ARCANE	67
6.2	Règles de décision fondées sur le score ML composite	67
7.1	Comparaison des métriques avant et après ajustement du pipeline ML . . .	71
8.1	Architecture de déploiement du système ARCANE	75
8.2	Séquence d'intégration entre Playwright, Flask et le moteur ML	75
8.3	Vue fonctionnelle du tableau de bord de supervision	77

Liste des sigles et acronymes

IA	Intelligence artificielle
ML	Machine Learning
NLP	Natural Language Processing
CNN	Convolutional Neural Network
LLM	Large Language Model
TF-IDF	Term Frequency–Inverse Document Frequency
CRISP-DM	Cross-Industry Standard Process for Data Mining
SSIM	Structural Similarity Index Measure
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MRR	Mean Reciprocal Rank
MTTR	Mean Time To Repair
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
DOM	Document Object Model
UI	User Interface
E2E	End-to-End
API	Application Programming Interface
REST	Representational State Transfer
REST-API	Representational State Transfer API
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
CORS	Cross-Origin Resource Sharing
ARIA	Accessible Rich Internet Applications
QA	Quality Assurance
CI-CD	Continuous Integration / Continuous Deployment
SaaS	Software as a Service
UML	Unified Modeling Language
FR	Français
EN	English
EPAM	Enterprise Product Application Management

Introduction

Les tests automatisés sont essentiels à la qualité des applications web, mais ils restent fragiles lorsque l'interface évolue. Un simple changement de DOM, d'attribut ou de libellé peut casser un sélecteur et générer un faux échec coûteux à maintenir.

Ce projet, réalisé chez **Moonside Consulting & Services**, propose un framework de *self-healing* capable d'identifier automatiquement un élément de remplacement. Il combine des signaux textuels, structurels, sémantiques et visuels afin de recommander un locator fiable avec un score de confiance.

Le rapport présente le contexte du projet, l'état de l'art, la démarche CRISP-DM, la préparation des données, la modélisation, l'évaluation et le déploiement du prototype. Il se termine par une synthèse des apports et des perspectives d'amélioration.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce chapitre présente le cadre général du projet réalisé chez **Moonside Consulting & Services**. Il résume l'organisme d'accueil, le contexte des tests automatisés, la problématique des sélecteurs fragiles et la solution de *self-healing* proposée.

1.2 Présentation de l'organisme d'accueil

1.2.1 Carte d'identité

Moonside Consulting & Services est une société de conseil en technologies de l'information fondée en 2021 aux Pays-Bas. Opérant depuis trois pays — Pays-Bas, France et Tunisie — elle accompagne ses clients dans leur transformation digitale, depuis la définition de la stratégie numérique jusqu'à l'exploitation et l'évolution des systèmes d'information. Sa philosophie d'intervention est résumée par la devise : « *Clarity that moves you forward* ».

Le tableau 1.1 présente les informations clés de l'entreprise.



FIGURE 1.1 – Logo de Moonside Consulting & Services

TABLE 1.1 – Carte d’identité de l’organisme d’accueil

Raison sociale	Moonside Consulting & Services
Forme juridique	Société de conseil en technologies et transformation digitale
Date de création	2021
Siège social	Pays-Bas (<i>présence opérationnelle en France et en Tunisie</i>)
Secteur d’activité	Conseil IT, développement logiciel, assurance qualité, transformation digitale, Data & IA
Certifications	ISO 27001 — Sécurité de l’information Partenaire officiel Odoo
Modèles de service	Consulting · Team Augmentation · Managed Services · Research & Innovation
Effectif	Équipe en croissance — ingénieurs, consultants et experts métier
Site web	https://www.moonside-cs.com/

1.2.2 Métiers de Moonside Consulting & Services

L’offre de Moonside Consulting & Services s’organise en deux grandes familles de services, couvrant à la fois les dimensions technologiques et opérationnelles des besoins de ses clients.

Services Technologie & Produit Ce premier pôle regroupe l’ensemble des prestations à forte composante technique :

- **Tech & Product Strategy & Transformation** : définition de feuilles de route technologiques, sélection de plateformes et conduite du changement.
- **Développement logiciel et QA** : conception et réalisation d’applications web robustes, assurance qualité — tests manuels et automatisés — et amélioration continue. *C’est dans ce domaine que s’inscrit directement le présent projet.*
- **Implémentation Odoo** : déploiements ERP sur-mesure en qualité de partenaire officiel Odoo, de l’installation à la maintenance post-déploiement.
- **Infrastructure & Cybersécurité** : conception d’architectures résilientes, renforcement de la posture de sécurité et accompagnement vers la certification ISO 27001.
- **DevOps & Cloud Engineering** : mise en place de pipelines CI/CD, Infrastructure as Code et déploiement d’architectures cloud-native.
- **Data & Intelligence Artificielle** : migration et préparation de données, analytique avancée et développement de cas d’usage en IA.

Services Opérationnels Ce second pôle couvre les fonctions de support métier nécessaires au bon fonctionnement des organisations :

- **Business Processes Transformation** : refonte de processus métier pour accroître l'efficacité opérationnelle.
- **RH & Finance Operations** : support au recrutement, gestion administrative et coordination de la paie.
- **Reporting & Data Collection** : structuration des flux de collecte, curation de données et tableaux de bord pour la conformité réglementaire (*RegTech*).
- **Customer Support & Account Management** : gestion des relations clients et pilotage des comptes.

La figure 1.2 offre une vue synthétique de l'ensemble de ces services.

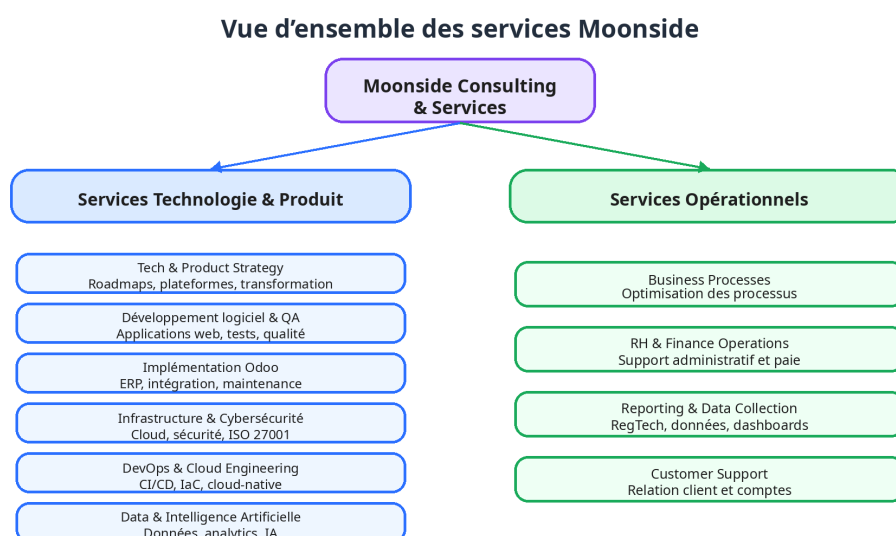


FIGURE 1.2 – Vue d'ensemble des services de Moonside Consulting & Services

Parmi les clients de référence figurent *Blackfin*, *CED*, *SmartTrade* et *Noveocare*, témoignant de la capacité de l'entreprise à intervenir dans des secteurs variés : Private Equity, Assurance, Services Financiers et Santé.

1.2.3 Organigramme

La structure organisationnelle de Moonside Consulting & Services repose sur quatre pôles complémentaires, articulés autour d'une direction générale. Cette organisation transversale favorise la collaboration entre les équipes techniques et les fonctions orientées métier.

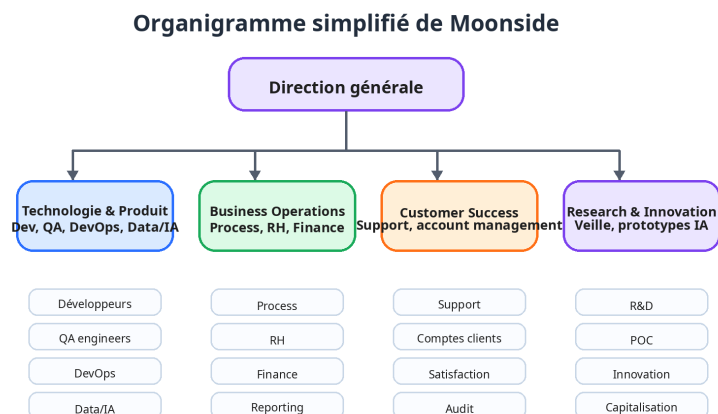


FIGURE 1.3 – Organigramme simplifié de Moonside Consulting & Services

Le stage s’est déroulé au sein du **Pôle Technologie & Produit**, qui regroupe les ingénieurs en développement logiciel, les spécialistes en assurance qualité, les experts DevOps ainsi que les équipes Data et IA. Ce cadre a permis de bénéficier d’une expertise pluridisciplinaire tout au long du projet.

1.2.4 Valeurs fondamentales

L’action de Moonside Consulting & Services est guidée par cinq valeurs fondatrices qui structurent ses engagements vis-à-vis de ses clients et de ses collaborateurs. Ces valeurs sont décrites dans le tableau 1.2.

TABLE 1.2 – Valeurs fondamentales de Moonside Consulting & Services

Valeur	Description
Innovation	Intégration continue des technologies modernes — IA, Cloud, DevOps — afin de proposer des solutions performantes et évolutives.
Qualité	Engagement à livrer des produits fiables, testés et conformes aux exigences des clients, renforcé par la certification ISO 27001.
Collaboration	Co-construction des solutions en étroite coopération avec les clients, selon un mode consulting embarqué ou en équipes dédiées.
Excellence technique	Application rigoureuse des meilleures pratiques en ingénierie logicielle, automatisation et architecture cloud.
Orientation client	Priorité donnée à la création de valeur mesurable pour chaque client, du conseil stratégique jusqu’au support opérationnel.

1.3 Contexte du projet

1.3.1 Cadre du projet

La qualité logicielle constitue un enjeu stratégique croissant dans le développement des applications modernes. En environnement agile — caractérisé par des cycles de livraison courts et des mises en production fréquentes — les tests automatisés sont devenus le principal mécanisme de détection des régressions et de validation du comportement attendu des systèmes.

Des frameworks spécialisés tels que **Selenium WebDriver**, **Cypress** et **Playwright** permettent aujourd’hui de couvrir de manière systématique les interfaces web. Leur fonctionnement repose sur des **sélecteurs** — expressions CSS, XPath, ou locators Playwright — qui désignent précisément chaque élément du Document Object Model (DOM) avec lequel le test doit interagir.

La figure 1.4 illustre le cycle de vie d’un test automatisé et met en évidence le rôle central du sélecteur dans la chaîne d’exécution.

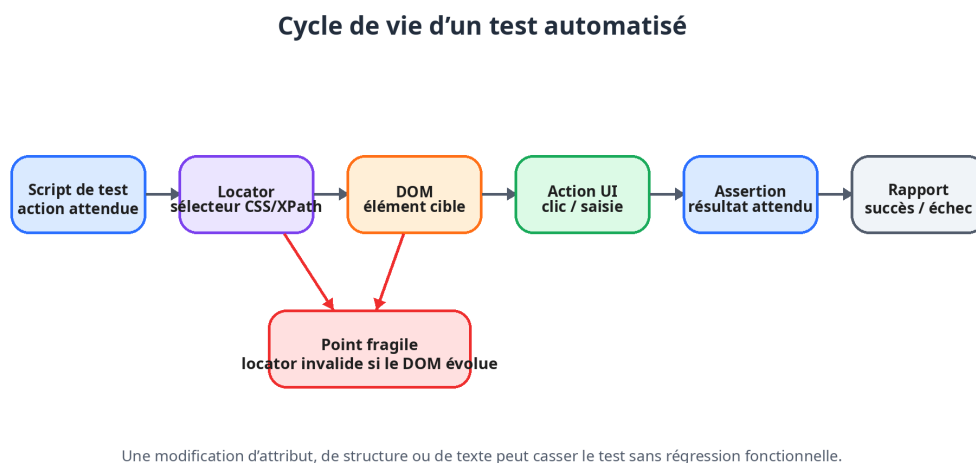


FIGURE 1.4 – Cycle de vie d’un test automatisé et point de fragilité des sélecteurs

Or, le sélecteur constitue également le maillon le plus fragile de cette chaîne. Toute évolution de l’interface — refonte graphique, migration de framework front-end, renommage de composants ou restructuration du DOM — peut invalider un sélecteur et provoquer l’échec du test correspondant. Ces échecs surviennent indépendamment de toute régression fonctionnelle : le système testé se comporte correctement, mais le test ne parvient plus à localiser l’élément cible.

1.3.2 Problématique

Ce phénomène de *sélecteur cassé* engendre une charge de maintenance considérable pour les équipes d’assurance qualité. Chaque mise à jour significative de l’interface peut nécessiter la révision manuelle de dizaines, voire de centaines de sélecteurs au sein de la suite de tests. Plus la couverture de tests est élevée, plus ce coût de maintenance est

important — créant ainsi un paradoxe : les projets les mieux testés sont aussi les plus pénalisés par les évolutions de l'interface.

La figure 1.5 illustre concrètement ce mécanisme de rupture, en comparant l'état du DOM avant et après une modification de l'interface et en montrant l'impact sur le sélecteur utilisé par le script de test.

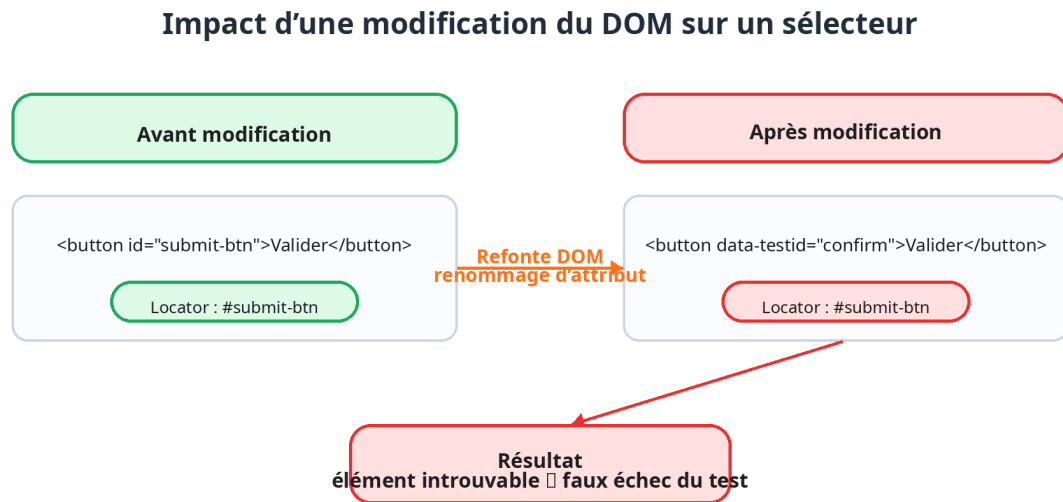


FIGURE 1.5 – Impact d'une modification du DOM sur la validité d'un sélecteur

Ce constat soulève une question centrale, qui constitue la problématique du présent projet :

Problématique : Comment identifier automatiquement, sans intervention humaine et en temps quasi réel, l'élément d'interface correspondant à un sélecteur devenu invalide suite à une évolution du DOM ?

Dans ce contexte, le *self-healing testing* vise à réparer automatiquement les échecs techniques liés à l'interface avant de conclure à une régression fonctionnelle. La problématique se résume ainsi :

- les applications web évoluent rapidement ;
- les tests E2E reposent sur des sélecteurs fragiles ;
- la réparation manuelle ralentit les équipes QA et CI/CD ;
- le projet vise une re-localisation automatique, contrôlée par un score de confiance et des traces vérifiables.

1.3.3 Objectifs et périmètre

Cette version est cadrée autour de trois axes :

TABLE 1.3 – Objectifs, périmètre et limites de la V1

Axe	Synthèse
Objectifs	Détecter un sélecteur invalide, retrouver l'élément candidat, générer des locators de remplacement, produire un rapport et mesurer le taux de réparation.
Périmètre V1	Applications web, tests E2E UI, actions élémentaires (clic, saisie, sélection, vérification), intégration Playwright et API HTTP/JSON compatible Selenium/Cypress/CI-CD.
Hors périmètre	Génération complète de scénarios, réparation d'assertions métier, correction des données de test et refontes graphiques sans signal stable.

1.3.4 Solution proposée

Pour répondre à cette problématique, le projet propose un **système de *self-healing* des tests automatisés**. Son principe est simple : analyser le DOM courant après un échec, identifier le meilleur candidat et proposer un sélecteur de remplacement sans interrompre l'exécution.

L'architecture retenue est hybride et s'appuie sur quatre couches d'analyse, illustrées en figure 1.6.

Architecture du système de self-healing proposé

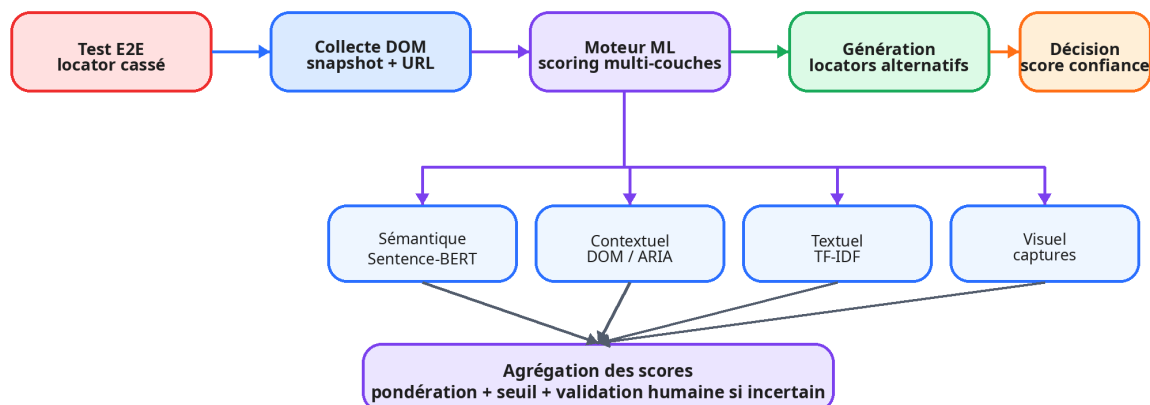


FIGURE 1.6 – Architecture du système de *self-healing* proposé

Le tableau 1.4 décrit le rôle de chacun de ces niveaux d’analyse.

TABLE 1.4 – Niveaux d’analyse du moteur de *self-healing*

Niveau d’analyse	Description
Sémantique	Comparaison du sens des éléments à l’aide de modèles de langue vectoriels (<i>embeddings</i> Sentence-BERT), permettant d’identifier des correspondances sémantiques même en l’absence de similarité syntaxique.
Contextuel	Analyse des attributs HTML et ARIA (<code>role</code> , <code>aria-label</code> , <code>data-testid</code> , etc.) pour caractériser la fonction de chaque élément dans la structure de la page.
Textuel	Comparaison du contenu textuel visible par l’utilisateur, en s’appuyant sur la pondération TF-IDF et la similarité cosinus entre vecteurs de termes.
Visuel	Comparaison des captures d’écran avant et après la modification afin de retrouver l’élément cible par sa position et son apparence visuelles.

Les scores sont agrégés dans un **score de confiance composite**. Si le meilleur score reste sous le seuil configuré, une validation humaine est requise.

- **Contribution** : analyse multi-sinaux et décision transparente.
- **Livrables** : framework, API CI/CD et protocole d’évaluation.
- **Métriques** : taux de réparation, MTTR, faux positifs et stabilité sur changements d’interface.

1.4 Planification et conduite du projet

1.4.1 Processus de développement

La conduite du projet s’est appuyée sur **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), retenue pour deux raisons principales :

- **itération** : ajuster les modèles à partir des résultats obtenus ;
- **orientation métier** : garder l’alignement entre choix techniques et objectifs QA.

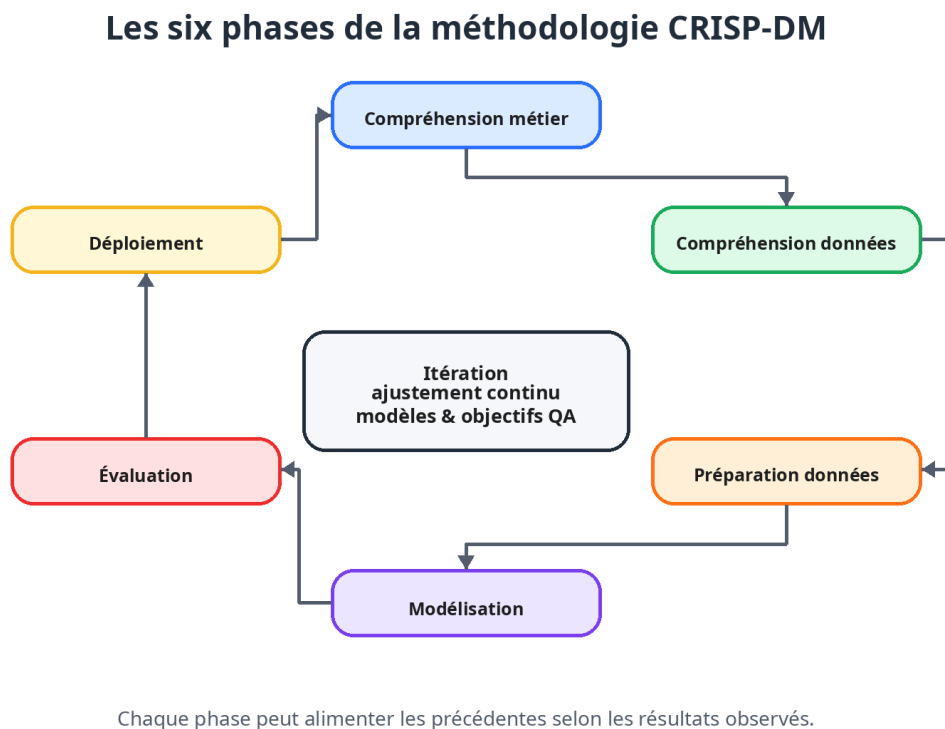


FIGURE 1.7 – Les six phases de la méthodologie CRISP-DM

Chaque cycle CRISP-DM part des résultats d’évaluation, ajuste les choix de modélisation puis met à jour les paramètres du moteur de scoring.

1.4.2 Planning du projet

Le projet était initialement planifié du 2 février au 31 juillet 2026. Grâce à l’avancement rapide des phases de modélisation, d’évaluation et de rédaction, les travaux ont été finalisés le 4 juillet 2026, soit avant l’échéance prévisionnelle. Les activités ont été organisées en six phases successives, chacune produisant un livrable identifié, comme le récapitule le tableau 1.5.

TABLE 1.5 – Planning synthétique du projet

Phase	Contenu	Livrable
1. Analyse des besoins	Recueil des exigences fonctionnelles et non fonctionnelles ; étude du problème de maintenance des sélecteurs.	Cahier des charges
2. État de l'art	Revue des solutions de <i>self-healing</i> existantes ; étude des frameworks de tests et des modèles NLP/Vision.	Rapport bibliographique
3. Conception	Définition de l'architecture du système ; choix technologiques ; modélisation UML des composants.	Diagrammes UML
4. Implémentation	Développement des quatre couches d'analyse et du moteur de scoring.	Code source
5. Tests & Validation	Évaluation sur jeux de données réels ; mesure de la précision, du rappel et du score F1.	Rapport d'évaluation
6. Rédaction	Rédaction du rapport de stage et préparation de la soutenance.	Rapport final

Le diagramme de Gantt présenté en figure 1.8 illustre le déroulement chronologique de ces phases et les éventuels chevauchements entre activités parallèles.

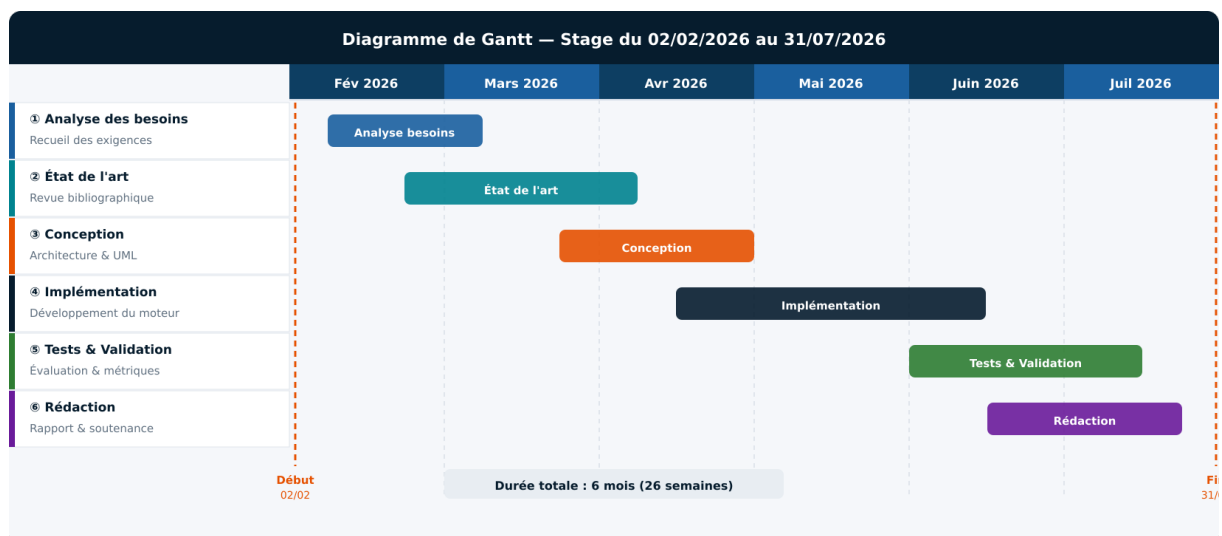


FIGURE 1.8 – Diagramme de Gantt — planning prévisionnel du stage (02/02–31/07/2026) et clôture effective le 04/07/2026

1.5 Conclusion

Ce chapitre a situé le projet dans son cadre professionnel et technique. Il a montré que la fragilité des sélecteurs justifie une solution de *self-healing* hybride, structurée selon CRISP-DM et fondée sur plusieurs couches d'analyse.

Chapitre 2

État de l’art et solution proposée

2.1 Introduction

Ce chapitre présente les principales approches de *self-healing* des tests automatisés et leurs limites. Il introduit ensuite la solution proposée, son architecture, son pipeline de décision et son interface d’intégration.

2.2 État de l’art — Outils et approches de *self-healing*

2.2.1 Panorama général

Le *self-healing* des tests automatisés désigne la capacité d’un système à détecter et corriger automatiquement les sélecteurs devenus invalides, sans intervention humaine. Deux grandes familles d’approches se dégagent :

- **Approche par snapshot** : enregistre les propriétés de chaque élément à la création du test et s’en sert pour le retrouver lors d’un échec.
- **Approche par intelligence artificielle** : mobilise des modèles d’apprentissage automatique — NLP, vision ou combinaison des deux — pour raisonner sur le sens et l’apparence des éléments.

La figure 2.1 cartographie les principaux outils recensés selon ces deux axes.

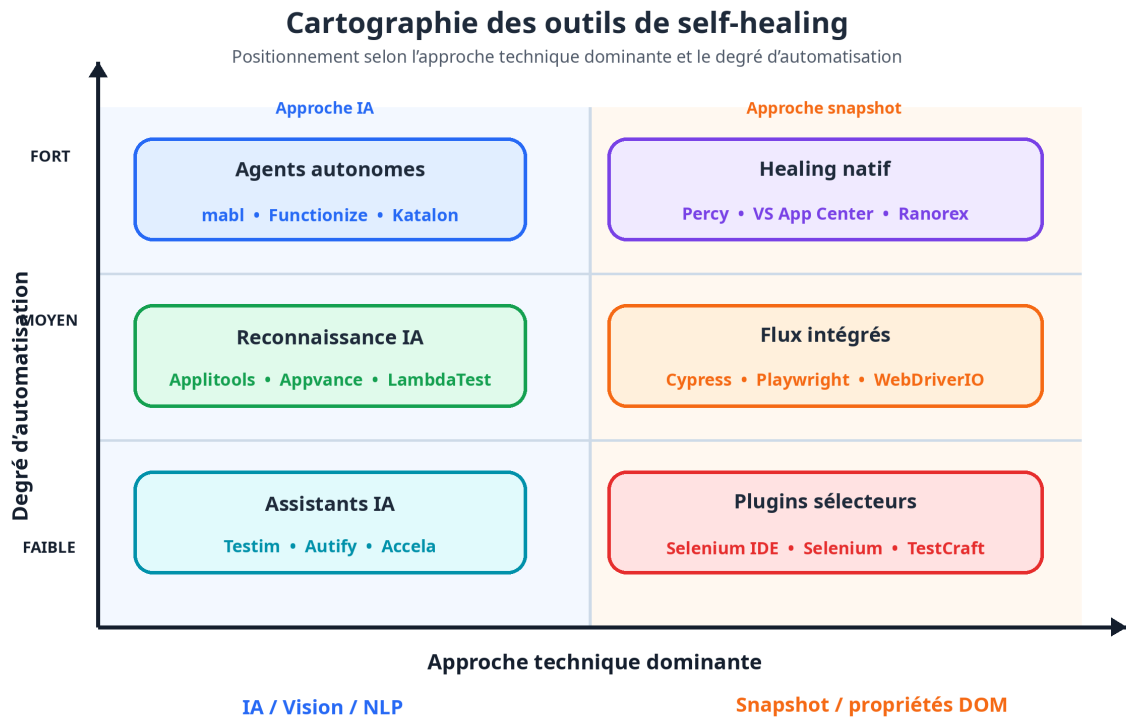


FIGURE 2.1 – Cartographie des outils de *self-healing* — positionnement selon l'approche technique

2.2.2 Healenium



FIGURE 2.2 – Logo de Healenium

Healenium (EPAM Systems) est une bibliothèque open source s'intégrant directement dans Selenium et Playwright. Lors de la création du test, il enregistre pour chaque sélecteur un arbre représentant la hiérarchie des nœuds DOM environnants ; en cas d'échec, il recherche la structure la plus proche par un score de similarité structurelle. Son intégration est transparente : seule la déclaration du pilote change, sans modifier les tests existants. Les corrections sont journalisées dans une base PostgreSQL et consultables via un tableau de bord web.

Forces. Très efficace sur les modifications syntaxiques légères (renommage d'identifiant, déplacement dans la hiérarchie DOM). Entièrement gratuit et open source.

Limites. Inefficace face aux changements sémantiques (libellé de bouton modifié, type d'élément changé). Absence de score de confiance interprétable. Requiert une instance PostgreSQL dédiée.

2.2.3 Testim



FIGURE 2.3 – Logo de Testim

Testim (acquis par Tricentis en 2022) est une plateforme commerciale dont le *self-healing* repose sur un modèle de classification supervisé : à chaque enregistrement, un vecteur multi-attributs est extrait pour chaque élément (id, classes, texte, position, rôle ARIA, voisinage DOM), et le modèle prédit l'élément le plus probable lors des exécutions futures. Le modèle est auto-adaptatif : il se réentraîne à mesure que l'équipe valide les corrections suggérées.

Forces. Niveau d'automatisation élevé, interface de création entièrement visuelle, rapports détaillés et gestion des environnements intégrée.

Limites. Modèle entièrement propriétaire, coût élevé, hébergement cloud (problèmes de confidentialité). La nature de boîte noire du modèle ML rend les corrections difficiles à expliquer, et le *lock-in* technologique est fort.

2.2.4 mabl



FIGURE 2.4 – Logo de mabl

mabl (fondé en 2017) repose sur un modèle ML qui analyse les attributs visuels et structurels des éléments pour maintenir les associations entre actions de test et éléments cibles. Sa caractéristique distinctive est la **détection proactive des anomalies** : la

plateforme surveille les tendances d'évolution de l'interface entre exécutions et signale les éléments dont la stabilité se dégrade, permettant d'anticiper les ruptures.

Forces. Excellente intégration CI/CD native (GitHub Actions, Jenkins, CircleCI), capacités de tests de performance et d'accessibilité intégrées.

Limites. SaaS propriétaire avec les mêmes contraintes que Testim sur le coût, la confidentialité et l'impossibilité de déploiement en environnement air-gapped.

2.2.5 AppliTools



FIGURE 2.5 – Logo de AppliTools

AppliTools adopte une approche radicalement différente : son moteur *Visual AI* analyse les captures d'écran via un réseau de neurones convolutifs (CNN) entraîné sur des millions d'interfaces. Lorsqu'un sélecteur échoue, l'élément est retrouvé par comparaison visuelle avec la référence enregistrée, indépendamment du DOM — ce qui en fait une solution idéale pour les frameworks générant des identifiants dynamiques.

Forces. Référence du marché pour les tests de régression visuelle. Tolère les variations mineures de rendu. S'intègre avec tous les principaux frameworks (Selenium, Playwright, Cypress, Appium).

Limites. Sensible aux changements de mise en page globaux (responsive design). Outil dédié à la validation visuelle, non fonctionnelle à part entière. Coût significativement plus élevé que les alternatives open source.

2.2.6 Cypress et Playwright natifs



FIGURE 2.6 – Logos de Cypress et Playwright

Ces deux frameworks n'offrent pas de *self-healing* natif, mais intègrent des mécanismes préventifs. Playwright privilégie les **locators sémantiques** (`get_by_role`, `get_by_label`) fondés sur les attributs ARIA, structurellement plus stables que les sélecteurs CSS. Il intègre également un mécanisme de *retry* automatique et des attentes intelligentes (*auto-waiting*). Cypress recommande les **data attributes** dédiés (`data-cy`, `data-testid`), stables par convention.

Limite commune. La robustesse offerte est purement préventive : si un attribut ARIA change ou si un composant est entièrement restructuré, le test échoue sans tentative de correction automatique.

2.2.7 Travaux académiques

WATER (Ceccato et al., 2010) est l'un des premiers systèmes formalisés de réparation de tests web. Il analyse le DOM avant et après une évolution via la distance d'édition entre arbres XML. Efficace sur les évolutions syntaxiques, mais limité par sa complexité computationnelle sur des pages de grande taille.

Vista (Leotta et al., 2016) propose une approche hybride combinant sélecteurs structurels, textuels et captures visuelles partielles, préfigurant l'idée de combinaison de signaux complémentaires au cœur de la présente solution.

SIDEREAL (Bajamonde et al., 2021) exploite le NLP pour analyser les descriptions sémantiques dans les commentaires du code source, montrant que les modèles de langage apportent une valeur ajoutée significative dans ce domaine.

Des travaux récents ont exploré l'utilisation de grands modèles de langage (LLM) pour suggérer des sélecteurs de remplacement. Si ces approches sont prometteuses, leur latence élevée et leur coût d'utilisation les rendent inadaptés à une intégration en pipeline CI/CD.

2.2.8 Tableau comparatif synthétique

TABLE 2.1 – Comparaison synthétique des outils et approches de *self-healing*

Outil / Ap- proche	Analyse sémantique	Analyse visuelle	Score	Open Source	CI/CD	Modif.
Healenium	Partielle	×	×	✓	✓	×
Testim	✓	Partielle	Implicite	×	✓	Partielle
mabl	✓	✓	Implicite	×	✓	Partielle
Applitools	×	✓	✓	×	✓	Partielle
PW natif	Préventif	×	×	✓	✓	×
WATER	×	×	×	✓	×	×
Vista	Partielle	✓	×	✓	×	Partielle
Solution	SBERT	SSIM	Explicite	Oui	API	Oui

2.2.9 Comparaison expérimentale des stratégies de résilience

Au-delà de l'état de l'art théorique, une comparaison expérimentale a été réalisée sur l'application de référence *Arcane Shop*. L'objectif n'est pas seulement d'opposer les outils entre eux, mais d'identifier le type de résilience apporté par chaque stratégie : réparation réactive, prévention par conception, assistance par agent IA ou validation visuelle. Cette distinction est importante, car deux approches peuvent obtenir de bons résultats tout en répondant à des besoins différents.

Le protocole compare quatre familles de solutions : le pipeline ML développé dans ce projet, des agents LLM de type GitHub Copilot, les locators natifs de Playwright et une approche de validation visuelle de type Applitools Eyes. Les tests portent sur des sélecteurs volontairement altérés dans une application e-commerce couvrant la navigation, les bannières, les fiches produits, la recherche, le panier, la wishlist et le checkout.

TABLE 2.2 – Positionnement expérimental des stratégies de self-healing

Stratégie	Nature	Infrastructure	Temps indicatif	Apport principal
Pipeline proposé	ML Réactive	Colab + Flask + ngrok	~48 s/test	Répare automatiquement les sélecteurs existants et fournit un score de confiance explicite.
Agents LLM	Réactive agentive	/ API LLM + prompts	30–60 s/test	Génère et adapte des tests avec une forte flexibilité, mais avec un coût et une latence élevés.
Locators Playwright natifs	Préventive	Playwright seul	5–15 s/test	Réduit fortement la casse grâce à <code>getByRole</code> , <code>getByText</code> et <code>getByTestId</code> .
Applitools Eyes	Visuelle	Cloud tools	10–20 s/test	Détecte les régressions visuelles et les écarts de rendu indépendamment du DOM.

Les résultats obtenus sur le pipeline ML montrent une performance élevée sur les cas de réparation étudiés. Sur 43 cas de test, le système obtient une accuracy de 95,3%, une précision de 94,3%, un rappel de 100% et un score F1 de 0,9706. Les huit cas négatifs sont utilisés pour vérifier la capacité du système à refuser une réparation lorsque l'élément attendu est absent ou que le sélecteur est manifestement non pertinent.

TABLE 2.3 – Résultats quantitatifs du pipeline ML sur les scénarios évalués

Métrique	Valeur	Interprétation
Cas évalués	43	Ensemble de sélecteurs cassés et cas négatifs.
Vrais positifs	33	Sélecteurs correctement réparés.
Faux positifs	2	Réparations proposées alors que le match reste discutable.
Vrais négatifs	8	Cas inexistants correctement rejetés.
Faux négatifs	0	Aucun élément guérissable manqué.
Accuracy	95,3%	Qualité globale de décision.
Précision	94,3%	Fiabilité des healings acceptés.
Rappel	100%	Capacité à retrouver tous les cas réparables.
F1 Score	0,9706	Compromis précision/rappel.

Cette comparaison met en évidence une complémentarité plutôt qu'une hiérarchie absolue. Les locators natifs Playwright constituent la meilleure stratégie préventive lorsqu'un nouveau projet peut être conçu avec de bonnes pratiques dès le départ. Le pipeline ML est plus adapté aux suites existantes ou aux applications legacy, car il peut réparer des sélecteurs déjà cassés et accélérer la migration vers des locators plus robustes. Les agents LLM sont pertinents pour l'assistance à la génération ou au debugging, mais leur coût et leur variabilité limitent leur usage systématique en CI/CD. Enfin, Applitools répond à une problématique complémentaire : la détection des régressions visuelles, qui ne remplace pas la réparation fonctionnelle des locators.

La stratégie la plus robuste apparaît donc hybride : utiliser les locators Playwright natifs pour prévenir la fragilité, mobiliser le pipeline ML pour le healing réactif des tests existants et compléter, lorsque le budget le permet, par une validation visuelle pour couvrir les changements de rendu.

2.2.10 Lacunes identifiées et positionnement de la solution

L'analyse comparative révèle trois lacunes communes qui justifient le développement d'une solution originale.

Absence de score de confiance explicite. La plupart des outils commerciaux opèrent leurs corrections de manière opaque. Un score numérique est pourtant indispensable pour décider si une correction peut être appliquée automatiquement ou si une validation humaine est nécessaire.

Mono-modalité de l'analyse. Aucun outil open source ne combine simultanément analyse sémantique par embeddings, attributs ARIA, pondération TF-IDF et comparaison visuelle SSIM.

Coût et dépendances cloud. Les solutions les plus avancées sont des plateformes SaaS incompatibles avec les environnements sécurisés ou les budgets contraints. La solution développée comble ce vide en exposant une API REST intégrable dans tout pipeline existant.

2.3 Motivation et principes de conception

2.3.1 Insuffisances des approches conventionnelles

Trois stratégies sont couramment utilisées en production, chacune avec ses limites structurelles, résumées dans le tableau 2.4.

La première, les **sélecteurs de secours manuels**, déplace le coût de maintenance sans le réduire : chaque évolution majeure oblige à réviser l'ensemble des chaînes de sélecteurs. La deuxième, le **snapshot d'attributs**, améliore la robustesse aux modifications syntaxiques mais reste vulnérable aux changements sémantiques (un bouton renommé de « Appliquer le filtre » à « Lancer la recherche » perd l'ensemble de ses propriétés enregistrées). La troisième, la **vision par ordinateur**, échoue dès que la mise en page évolue significativement.

TABLE 2.4 – Comparaison des stratégies conventionnelles de gestion des sélecteurs cassés

Approche	Avantages	Limites	Auto-matisation
Sélecteurs de secours manuels	Simple, sans outillage supplémentaire	Maintenance exponentielle avec la taille de la suite	Nulle
Snapshot d'attributs (Healenium)	Robuste aux changements syntaxiques	Échoue sur modifications sémantiques	Partielle
Vision par ordinateur	Indépendant du DOM	Sensible aux changements de mise en page	Partielle
Approche proposée (hybride)	Combine 4 signaux complémentaires	Coût computationnel non nul	Complète

2.3.2 Principes directeurs de la conception

Trois principes fondateurs gouvernent l'ensemble des choix architecturaux.

Complémentarité des signaux. Aucune source d'information n'est seule suffisante : un élément peut voir son texte modifié sans que son rôle ARIA change, ou migrer dans le DOM sans que son apparence soit altérée. La solution exploite donc quatre types de signaux dont les scores sont agrégés en un indicateur composite unique.

Réduction progressive. Une stratégie d'entonnoir élimine à chaque étape les candidats les moins plausibles, concentrant les analyses les plus coûteuses (SSIM) sur un nombre très limité d'éléments finalistes.

Confiance quantifiée et transparence. En deçà d'un seuil configurable, aucune correction silencieuse n'est appliquée : le système signale l'incertitude et sollicite une validation humaine, évitant ainsi le piège des corrections automatiques erronées.

2.4 Analyse des besoins

2.4.1 Contexte applicatif et scénarios de référence

L'application web cible est considérée comme une application métier classique composée de pages de connexion, de tableaux de bord, de formulaires de saisie, de listes filtrables et de composants interactifs tels que boutons, champs, menus déroulants et liens d'action. Afin de préserver la confidentialité du contexte industriel, les écrans sont anonymisés ; les mécanismes étudiés restent cependant représentatifs des interfaces web modernes développées avec des frameworks front-end évolutifs.

Trois scénarios E2E servent de référence pour la conception et l'évaluation : authentification puis accès au tableau de bord, recherche d'un élément dans une liste filtrable, et soumission d'un formulaire après modification de libellés ou d'attributs HTML. Ces scénarios couvrent les ruptures les plus fréquentes : renommage d'identifiants, déplacement d'un composant, changement de texte visible et modification d'attributs ARIA.

Les entrées du système sont les scripts de test, le sélecteur défaillant, l'URL de la page, les snapshots DOM/HTML, les logs d'exécution, les captures d'écran et, lorsque disponible, le locator ou le texte de référence. Les sorties sont une proposition de nouveaux locators, un score de confiance, un rapport de healing, les candidats alternatifs et les indicateurs nécessaires au calcul du taux de succès.

2.4.2 Identification des acteurs

Le système de *self-healing* interagit avec trois catégories d'acteurs, dont les rôles et interactions sont résumés dans le tableau 2.5.

TABLE 2.5 – Acteurs du système et leurs rôles

Acteur	Rôle	Type d'interaction
Ingénieur de test	Configure les paramètres du système (seuil de confiance, URL cible, texte de référence), consulte les résultats et valide les corrections en cas de score insuffisant.	Directe (API REST / interface)
Pipeline CI/CD	Invoke automatiquement l'endpoint <code>POST /heal</code> lors de la détection d'un sélecteur défaillant, consomme la réponse JSON et applique le locator recommandé dans la suite de tests.	Automatisée (HTTP)
Application web cible	Fournit le DOM et le rendu visuel analysés par le moteur. Son évolution est précisément la cause des sélecteurs cassés que le système cherche à corriger.	Passive (pilotage Playwright)

2.4.3 Besoins fonctionnels

Les besoins fonctionnels décrivent les capacités que le système doit offrir pour répondre aux objectifs du projet. Ils sont organisés en cinq groupes fonctionnels.

BF1 — Acq. et analyse de la page.

- **BF1.1** Le système doit charger la page web cible via Playwright en mode *headless* et en extraire l'ensemble des éléments interactifs visibles.
- **BF1.2** Le système doit capturer une capture d'écran pleine page afin d'alimenter l'analyse visuelle.
- **BF1.3** Pour chaque élément, le système doit récupérer les attributs structurels (`tag`, `id`, `class`, `name`, `placeholder`, `type`, `aria-label`) ainsi que le contenu textuel et les coordonnées.

BF2 — Prétrait. et représentation sémantique.

- **BF2.1** Le système doit décomposer le sélecteur défaillant en ses constituants sémantiques et en dériver un texte de référence normalisé.
- **BF2.2** Le système doit expander les abréviations techniques (ex. `filt` → `filter`) par comparaison avec un vocabulaire d'interface utilisateur de référence.
- **BF2.3** Le système doit proposer une traduction optionnelle français → anglais pour les sélecteurs rédigés en français.

BF3 — Pipeline de décision multi-modal.

- **BF3.1** Le système doit calculer un score de similarité sémantique entre le vecteur de référence et chaque élément candidat via Sentence-BERT et retenir les quinze meilleurs.
- **BF3.2** Le système doit enrichir les scores par une analyse des attributs ARIA et de la correspondance textuelle directe, puis réduire à huit candidats.
- **BF3.3** Le système doit affiner les scores par vectorisation TF-IDF et réduire à trois candidats finalistes.
- **BF3.4** Le système doit calculer un score SSIM entre la région visuelle de chaque candidat et l'image de référence, et produire un score composite pondéré (70 % linguistique, 30 % visuel).
- **BF3.5** Le système doit appliquer un mécanisme d'amplification sélective (+0,15) aux éléments partageant des fragments avec le sélecteur original.

BF4 — Persistance et gestion de la référence.

- **BF4.1** Le système doit enregistrer les propriétés de l'élément cible dans un fichier `baseline.json` lors de la première exécution.
- **BF4.2** Le système doit sauvegarder la capture visuelle de l'élément de référence dans `baseline.png`.

- **BF4.3** Le système doit tenter de retrouver l'élément de référence en cascade (correspondance directe, validation SSIM, scan visuel global, sélection sémantique fraîche) lors des exécutions suivantes.

BF5 — Génération de locators et exposition API.

- **BF5.1** Le système doit générer une liste ordonnée de locators Playwright selon huit stratégies classées par stabilité décroissante.
- **BF5.2** Le système doit sérialiser les résultats en JSON (`locators.json`) avec le locator recommandé, l'ensemble des alternatives, le score de confiance et les métadonnées de l'élément.
- **BF5.3** Le système doit exposer un endpoint `GET /ping` retournant l'état opérationnel du service.
- **BF5.4** Le système doit exposer un endpoint `POST /heal` acceptant un sélecteur défaillant, une URL cible, un texte de référence optionnel et un seuil de confiance configurable.
- **BF5.5** En deçà du seuil de confiance, le système doit émettre une alerte explicite et solliciter une validation humaine plutôt que d'appliquer silencieusement une correction.

2.4.4 Besoins non fonctionnels

TABLE 2.6 – Besoins non fonctionnels du système

ID	Catégorie	Description	Critère mesurable
BNF1	Performance	Le pipeline complet (acquisition + 4 étapes + génération) doit s'exécuter dans un délai compatible avec une intégration CI/CD interactive. Le choix de <code>all-MiniLM-L6-v2</code> (22 M de paramètres) versus des modèles plus lourds est motivé par cette contrainte.	≤ 30 s
BNF2	Fiabilité	Le système doit produire un résultat déterministe pour une même paire (sélecteur, URL). Le score composite doit refléter fidèlement la confiance réelle de la correction proposée.	Score $\in [0, 1]$
BNF3	Maintenabilité	Chaque module (acquisition, prétraitement, pipeline, API) doit être indépendant et remplaçable sans modifier les autres. Les poids du score composite et le seuil de confiance doivent être configurables sans recompilation.	Paramètres externalisés
BNF4	Interopérabilité	L'API REST doit être invocable depuis n'importe quel framework de test (Pytest, Playwright JS, Cypress via plugin, Jenkins, GitHub Actions) sans dépendance directe au code Python du moteur.	HTTP/JSON
BNF5	Portabilité	Le système doit fonctionner dans un environnement contraint (Google Colab, CPU seul) sans nécessiter d'infrastructure dédiée. Il doit être déployable sur tout serveur Linux exposant un port HTTP.	CPU seul / Linux
BNF6	Sécurité	Les échanges entre le client et l'API doivent transiter en HTTPS. Aucune donnée sensible de l'application testée ne doit être persistée.	HTTPS
BNF7	Transparence	Toute décision de correction doit être accompagnée d'un score numérique explicite et d'une liste de locators alternatifs classés pour audit.	Score + liste
BNF8	Scalabilité	Le système doit pouvoir traiter plusieurs requêtes concurrentes sans partage d'état mutable entre requêtes.	Async / multi-thread

2.4.5 Diagramme des cas d'utilisation

Le diagramme 2.7 synthétise les interactions entre les acteurs et le système. Trois cas d'utilisation principaux sont identifiés :

- **Réparer un sélecteur** (*use case* central) : déclenché par le pipeline CI/CD ou l'ingénieur, il enchaîne acquisition, pipeline de décision et génération de locators.
- **Configurer le système** : réservé à l'ingénieur de test, il couvre le paramétrage du seuil de confiance, de l'URL et du texte de référence.
- **Valider une correction ambiguë** : sollicité uniquement lorsque le score composite est inférieur au seuil, il implique une décision humaine avant application.

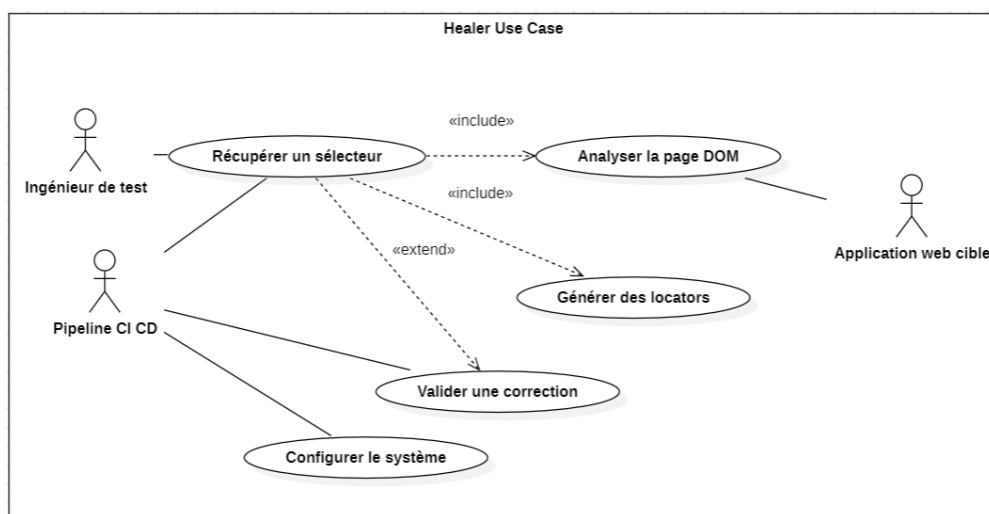


FIGURE 2.7 – Diagramme des cas d'utilisation — acteurs et interactions principales

2.4.6 Contraintes et hypothèses de conception

Trois contraintes structurelles ont conditionné l'ensemble des choix architecturaux présentés dans ce chapitre.

Contrainte d'environnement. Le moteur doit fonctionner sur Google Colab avec un runtime CPU standard (12 Go de RAM partagée). Cette contrainte exclut les modèles de vision lourds et justifie le choix de SSIM — algorithme signal plutôt que réseau de neurones — pour l'analyse visuelle.

Contrainte d'intégration non intrusive. Le système ne doit requérir aucune modification des tests existants : seule l'URL de l'API doit être configurée côté client. Cette exigence a dicté l'architecture découplée autour de l'API REST.

Hypothèse de stabilité relative. Le système suppose que, lors d'une évolution d'interface, au moins l'un des quatre signaux (sémantique, ARIA, lexical, visuel) reste partiellement stable. Cette hypothèse est cohérente avec les pratiques d'évolution progressive

des interfaces web modernes, mais peut être mise en défaut lors de refontes graphiques totales.

Baseline et garde-fous. L'évaluation compare une baseline sans *self-healing*, qui échoue dès qu'un sélecteur ne correspond plus, à une baseline heuristique simple fondée sur les attributs stables (`id`, `name`, `data-testid`, XPath/CSS proches) puis à l'approche multi-signaux proposée. La réparation n'est acceptée que si le score composite dépasse un seuil de confiance, si le test repasse après substitution du locator et si l'assertion métier associée reste valide. Toute proposition est journalisée afin de permettre l'audit et d'éviter les corrections silencieuses.

Les métriques figées pour le chapitre d'évaluation sont le taux de réparations automatiques réussies, le top-1/top-5 des candidats proposés, le MRR (*Mean Reciprocal Rank*), le temps moyen de réparation (MTTR), le taux de faux positifs et la stabilité sur plusieurs séries de changements UI.

2.5 Architecture générale du système

2.5.1 Vue d'ensemble des composants

Le système repose sur quatre modules principaux illustrés en figure 2.8, chacun à responsabilité délimitée.

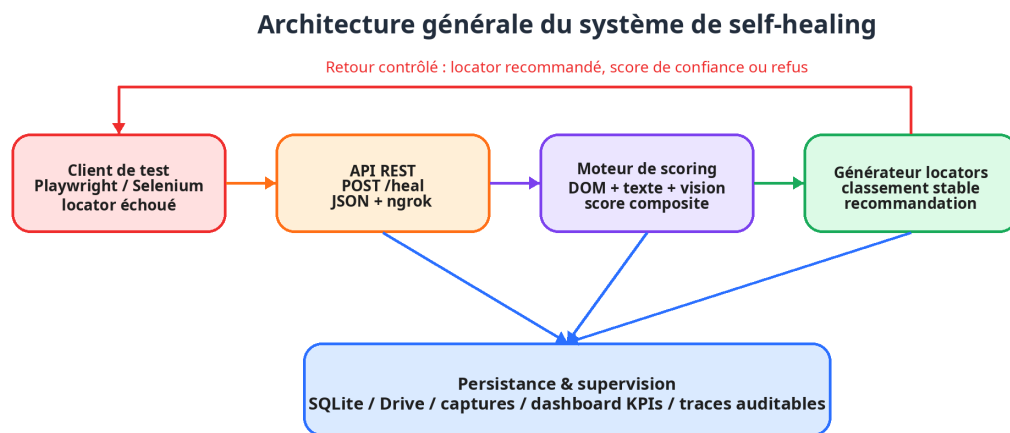


FIGURE 2.8 – Architecture générale du système de *self-healing* — quatre modules principaux

Module d'acquisition. S'appuie sur Playwright en mode *headless* pour charger la page cible, extraire les éléments interactifs visibles du DOM et capturer une capture d'écran pleine page. Pour chaque élément retenu, il récupère : `tag`, `id`, `class`, `name`, `placeholder`, `type`, `aria-label`, contenu textuel et coordonnées. Le sélecteur cible :

`button, input, a, select, textarea, label, div[role], span[role]`

Module de prétraitement linguistique. Prépare la représentation textuelle de référence en trois passes : (1) décomposition syntaxique du sélecteur en ses constituants (tag, id, classes, attributs) ; (2) expansion des abréviations techniques via Sentence-BERT comparé à un vocabulaire de référence d’une centaine de termes UI courants ; (3) traduction optionnelle FR→EN via *Helsinki-NLP/opus-mt-fr-en*.

Moteur de pipeline. Cœur analytique du système, il implémente la séquence de quatre étapes décrite à la section 2.6.

Module de génération de locators. Prend en entrée l’élément retenu et produit une liste ordonnée de locators Playwright selon huit stratégies de localisation classées par stabilité décroissante.

2.5.2 Mécanisme de persistance de la référence

Lors de la première exécution, le système sauvegarde les propriétés de l’élément cible dans `baseline.json` (attributs structurels) et `baseline.png` (capture de la région). Lors des exécutions suivantes, il tente de retrouver cet élément en cascade :

1. **Correspondance directe** (identifiant ou tag + classe + texte) revalidée par score sémantique $\geq$ seuil de confiance.
2. **Validation visuelle SSIM** : comparaison de la région capturée à `baseline.png` ; accepté si SSIM $> 0,50$.
3. **Scan visuel global** : SSIM sur tous les éléments visibles ; meilleur correspondant retenu si SSIM $> 0,40$.
4. **Sélection sémantique fraîche** : meilleur score cosinus, sans contrainte de seuil.

2.5.3 Flux de traitement global

Le diagramme de séquence figure 2.9 représente le flux complet du sélecteur défaillant jusqu’au locator recommandé. Le traitement s’effectue dans une boucle *asyncio* dédiée par requête, assurant l’isolation entre appels concurrents.

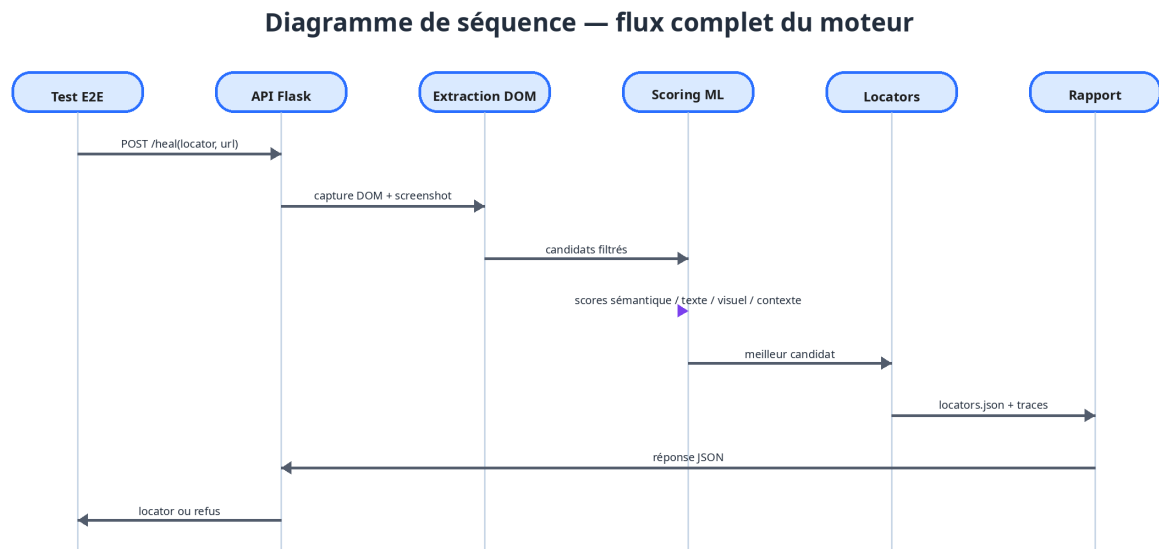


FIGURE 2.9 – Diagramme de séquence — flux de traitement complet du moteur de *self-healing*

2.6 Pipeline de décision en quatre étapes

Le pipeline reçoit la liste complète des éléments DOM et le vecteur d’embedding de référence, et produit un classement des candidats avec score de confiance composite. Son fonctionnement en entonnoir est illustré en figure 2.10.

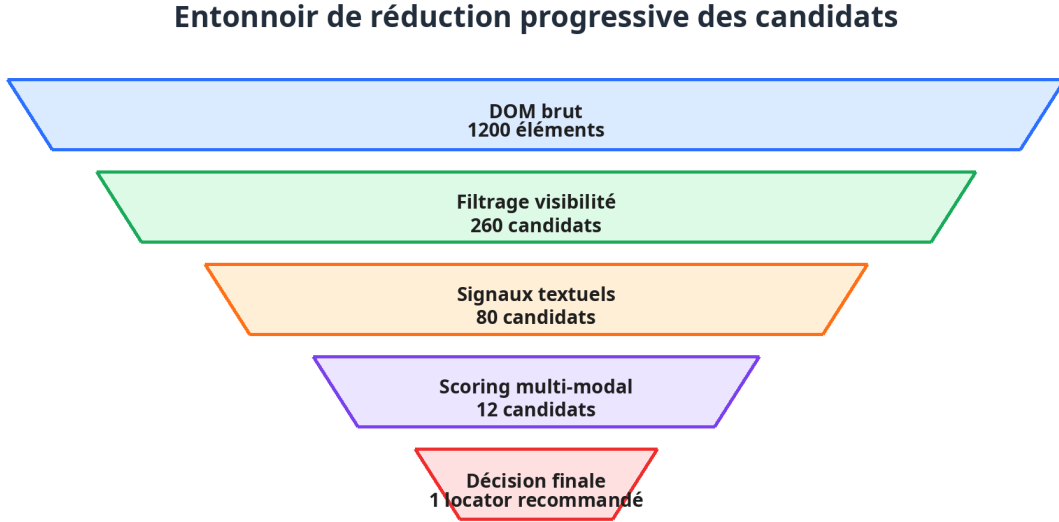


FIGURE 2.10 – Entonnoir de réduction progressive des candidats à travers les quatre étapes

2.6.1 Étape 1 — Analyse sémantique par Sentence-BERT

L’analyse sémantique est la première et la plus discriminante des quatre étapes. Elle repose sur **Sentence-BERT** (all-MiniLM-L6-v2, 384 dimensions), modèle spécifiquement entraîné à la tâche de similarité sémantique de phrases, retenu pour ses performances sur les benchmarks STSB/NLI et sa faible latence d’inférence (22 M de paramètres).

Chaque élément est converti en chaîne normalisée par concaténation de ses attributs descriptifs :

$$t(e) = \text{preprocess}(\text{text}_e \parallel \text{id}_e \parallel \text{class}_e \parallel \text{name}_e \parallel \text{placeholder}_e \parallel \text{tag}_e \parallel \text{type}_e) \quad (2.1)$$

Le score sémantique est défini par la similarité cosinus des embeddings :

$$s_{\text{sem}}(e, r) = \frac{\phi(t(e)) \cdot \phi(t(r))}{\|\phi(t(e))\| \cdot \|\phi(t(r))\|} \quad (2.2)$$

Un mécanisme d’**amplification sélective** (+0,15) est appliqué aux éléments partageant des fragments avec le sélecteur original. Les **quinze** candidats au score le plus élevé sont transmis à l’étape suivante.

2.6.2 Étape 2 — Analyse contextuelle par attributs ARIA

Cette étape enrichit les scores en exploitant les attributs HTML et ARIA, plus stables que les identifiants ou classes lors des évolutions d’interface. Trois contributions booléennes sont évaluées :

1. **Correspondance de tag** (δ_{tag} , poids 0,30) : le tag HTML correspond au type inféré depuis le sélecteur de référence.
2. **Correspondance textuelle directe** (δ_{text} , poids 0,70) : le texte normalisé de référence est contenu dans le texte visible de l’élément.
3. **Correspondance ARIA** (δ_{aria} , poids 0,40) : l’`aria-label` contient le texte de référence normalisé.

$$s_{\text{ctx}}(e) = s_{\text{sem}}(e) + 0,3 \times (\delta_{\text{tag}} \cdot 0,30 + \delta_{\text{text}} \cdot 0,70 + \delta_{\text{aria}} \cdot 0,40) \quad (2.3)$$

Les **huit** meilleurs candidats sont transmis à l’étape suivante.

2.6.3 Étape 3 — Analyse textuelle par TF-IDF

Le TF-IDF identifie des **correspondances lexicales précises**, valorisant les termes rares et distinctifs. Il est complémentaire aux embeddings de l’étape 1 : un fort score sémantique sans correspondance lexicale peut signaler une correspondance ambiguë, et inversement. Les textes des huit candidats et la référence sont vectorisés conjointement via `TfidfVectorizer` (scikit-learn) :

$$s_{\text{txt}}(e) = s_{\text{ctx}}(e) + 0,4 \times \cos(\mathbf{v}_{\text{tfidf}}(e), \mathbf{v}_{\text{tfidf}}(r)) \quad (2.4)$$

Les **trois** meilleurs candidats sont transmis à l’étape visuelle.

2.6.4 Étape 4 — Analyse visuelle par SSIM

Même après un remaniement complet des attributs HTML, l’apparence visuelle d’un élément (couleur, forme, position relative) peut demeurer stable. L’algorithme SSIM (Wang et al., 2004) évalue la similarité perceptuelle via trois composantes : luminance l , contraste c et structure s :

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (2.5)$$

La région de la capture pleine page correspondant à chaque candidat est découpée, redimensionnée aux dimensions de `baseline.png`, puis comparée en mode couleur. Le score final est une combinaison linéaire pondérée :

$$s_{\text{final}}(e) = 0,70 \times s_{\text{txt}}(e) + 0,30 \times \text{SSIM}(e, r) \quad (2.6)$$

La pondération 70/30 accorde la primauté aux signaux linguistiques (plus robustes aux variations de rendu) tout en donnant à la composante visuelle un rôle discriminant en cas d’ambiguïté. Le candidat de rang 1 est ensuite soumis à la règle de seuil θ_{conf} (défaut : 0,45) : en deçà, une alerte est émise et la validation humaine est sollicitée. La figure 2.11 illustre l’évolution des scores des trois finalistes.

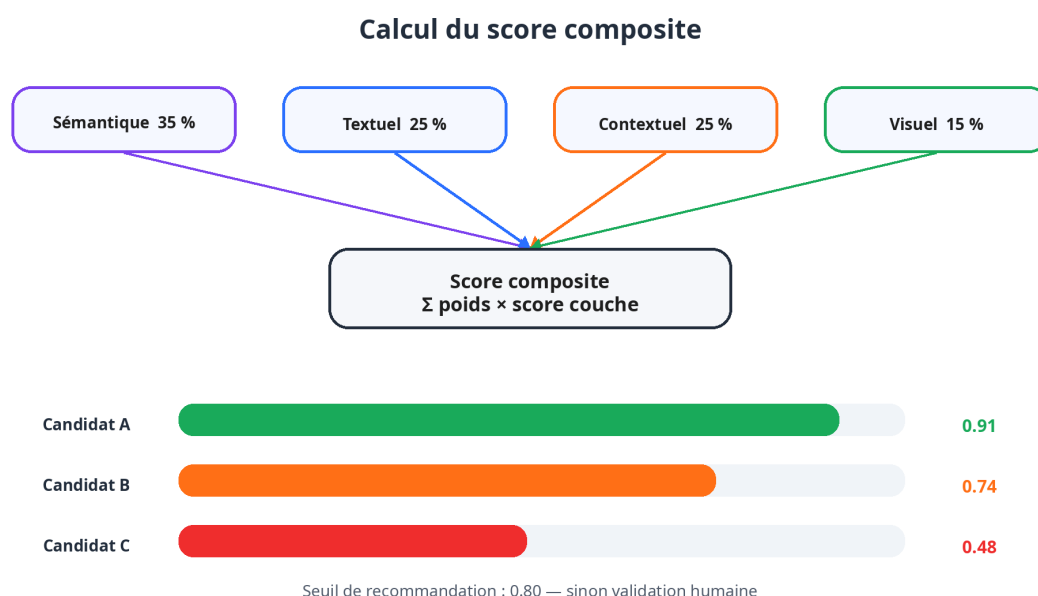


FIGURE 2.11 – Calcul du score composite — pondération des couches et évolution des scores des candidats

2.7 Génération des locators de remplacement

Le module de génération produit une liste ordonnée de locators Playwright, chacun accompagné d’un niveau de stabilité. La hiérarchie adoptée (figure 2.12) reflète la résistance attendue de chaque stratégie face aux évolutions futures : les attributs à vocation sémantique stable (`id`, `name`, `aria-label`) sont classés en tête ; le sélecteur de tag seul constitue le dernier recours.

Hiérarchie des locators générés

1	getByRole	page.getByRole("button", { name: "Valider" })	Très stable
2	getByLabel	page.getByLabel("Email")	Très stable
3	getByTestId	page.getByTestId("submit")	Stable
4	getByText	page.getByText("Valider")	Moyen
5	CSS robuste	page.locator("button[type=submit]")	Moyen
6	XPath/CSS fragile	page.locator("#root > div:nth-child(2)")	Faible

FIGURE 2.12 – Hiérarchie des locators générés — classement par stabilité avec exemples de code

Les résultats sont sérialisés en JSON (`locators.json`) dont la structure est décrite dans le tableau 2.7.

TABLE 2.7 – Structure du document JSON de sortie

Champ	Description
<code>element</code>	Métadonnées de l'élément identifié (tag, id, name, text, type, class, aria_label)
<code>element_position</code>	Coordonnées et dimensions dans la page (x, y, w, h)
<code>baseline_text</code>	Représentation textuelle dérivée du sélecteur original
<code>recommended</code>	Locator Playwright de rang 1 — directement intégrable
<code>all_locators</code>	Liste complète avec rang, type, stabilité et note explicative

2.8 Interface d'intégration : l'API REST

2.8.1 Conception

Le moteur est exposé via une **API REST** implémentée avec Flask, ce qui le découple de ses consommateurs : un script Pytest, une suite Playwright en JavaScript ou un pipeline Jenkins peuvent tous l'invoquer via une simple requête HTTP, sans partage de contexte d'exécution. La figure 2.13 représente les deux points de terminaison exposés.

Interface API REST — endpoints et intégration ngrok

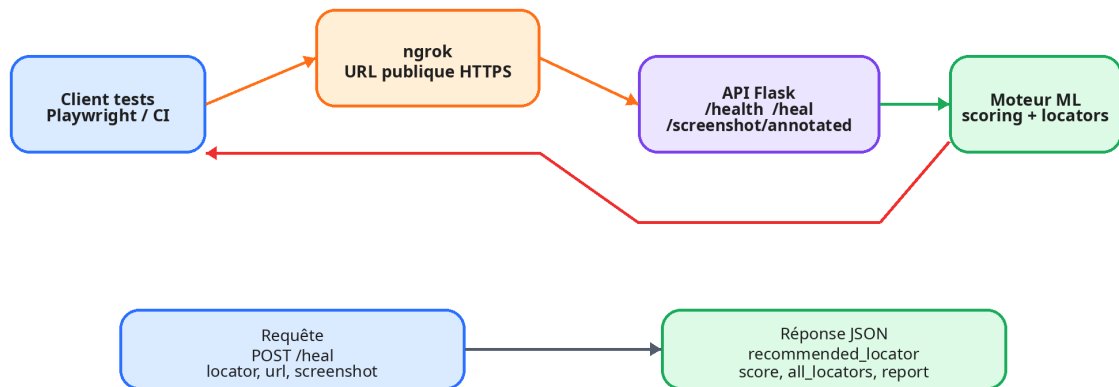


FIGURE 2.13 – Interface API REST — points de terminaison et flux d'intégration avec ngrok

GET /ping. *Health check* retournant l'état opérationnel du service, le modèle chargé, le sélecteur de référence actif et l'URL cible par défaut.

POST /heal. Reçoit le sélecteur défaillant, l'URL à analyser, un texte de référence alternatif optionnel et le seuil de confiance. Déclenche le pipeline complet (dans une boucle `asyncio` isolée) et retourne le locator recommandé, l'ensemble des locators alternatifs, le score de confiance et les métadonnées de l'élément identifié.

TABLE 2.8 – Paramètres et réponse de l’endpoint POST /heal

Paramètre	Description	Type	Requis
<i>Corps de la requête (JSON)</i>			
<code>broken_selector</code>	Sélecteur CSS, XPath ou locator Playwright défaillant	string	Oui
<code>url</code>	URL de la page à analyser	string	Non
<code>baseline_text</code>	Texte de référence alternatif pour l’embedding	string	Non
<code>confidence_threshold</code>	Seuil de confiance minimal (défaut : 0,45)	float	Non
<i>Corps de la réponse (JSON)</i>			
<code>success</code>	Indicateur de succès du traitement	bool	—
<code>recommended</code>	Locator Playwright de rang 1	string	—
<code>score</code>	Score de confiance composite final	float	—
<code>all_locators</code>	Liste complète des locators	array	—
<code>element</code>	Métadonnées de l’élément identifié	object	—
<code>baseline_derived</code>	Représentation textuelle dérivée	string	—
<code>stage_counts</code>	Candidats après chaque étape	array	—

2.8.2 Gestion des requêtes et déploiement

Chaque appel /heal s’exécute dans une boucle `asyncio` isolée, évitant tout partage d’état mutable entre requêtes concurrentes. Flask est configuré en mode multi-threadé (`threaded=True`) et la gestion CORS est assurée par `flask-cors`.

Dans le cadre expérimental sous Google Colab, l’API est exposée sur internet via `ngrok`, associant le port local 5000 à une URL HTTPS publique communiquée via `COLAB_API_URL`. Cette architecture reflète un scénario réel : moteur déployé sur un serveur centralisé, invoqué par des agents de test distribués.

2.9 Choix technologiques

Les choix ont été guidés par trois critères : qualité des résultats, compatibilité avec un déploiement CPU contraint (Google Colab) et maturité de l’écosystème Python. Le tableau 2.9 présente la stack retenue.

TABLE 2.9 – Stack technologique complète du système de *self-healing*

Logo	Technologie	Rôle	Version	Couche
 [1]	Playwright (Python)	Pilotage Chromium headless, extraction DOM, screenshot	async API	Acq.
	Sentence-BERT all-MiniLM-L6-v2 [2]	Encodage sémantique 384 dim., similarité cosinus	2.6.1 / 22 M paramètres	Sém.
	scikit-learn Tfidf Vectorizer [3, 4]	Vectorisation TF-IDF, similarité cosinus lexicale	1.x	Texte
	OpenCV + scikit-image [5, 6]	Traitement d'image, redimensionnement, calcul SSIM	—	Visuel
	Helsinki-NLP opus-mt-fr-en [7]	Traduction machine FR→EN (optionnel)	—	Prétrait.
	Flask + Flask-CORS [8]	Serveur REST de prototype/démo ; en production : gunicorn/uvicorn	—	API
	ngrok [9]	Tunneling HTTPS pour prototype Colab et démonstration distante	—	Dépl.
	NumPy	Opérations vectorielles, normalisation, agrégation	—	Sém.
	nest_asyncio	Autorisation des boucles asyncio imbriquées sous Jupyter/Colab	—	Dépl.

2.9.1 Justification du choix du modèle sémantique

Le module sémantique constitue le premier filtre intelligent du système : il doit rapprocher le sélecteur cassé des éléments DOM candidats même lorsque le texte, l'identifiant ou les classes CSS ont partiellement changé. Le choix du modèle ne peut donc pas être réduit à la seule précision brute. Dans le contexte du projet, il doit également respecter trois contraintes opérationnelles : une inférence rapide sur CPU, une empreinte mémoire compatible avec Google Colab et une intégration simple dans une pipeline Python appelée pendant l'exécution des tests.

Quatre familles d'approches ont été comparées : une similarité lexicale TF-IDF, un modèle Sentence-BERT léger (**all-MiniLM-L6-v2**), une variante Sentence-BERT plus lourde (**all-mpnet-base-v2**) et un modèle LLM généraliste accessible par API. Le tableau 2.10 synthétise l'analyse qualitative réalisée avant intégration.

TABLE 2.10 – Comparaison des approches candidates pour la similarité sémantique

Approche	Qualité	Latence	Dépl.	Limite principale
TF-IDF seul	Moyenne	Très faible	Très simple	Sensible aux reformulations, synonymes et changements de langue.
all-MiniLM-L6-v2	Élevée	Faible	Simple	Moins expressif que les modèles de grande taille sur des cas ambigus.
all-mpnet-base-v2	Très élevée	Moyenne	Simple	Temps d'encodage et mémoire plus coûteux pour un gain marginal.
LLM généraliste	Très élevée	Élevée	Complexe	Coût, dépendance réseau, variabilité et enjeux de confidentialité.

all-MiniLM-L6-v2 est retenu car il se situe dans la meilleure zone de compromis. Il produit des embeddings de 384 dimensions, suffisamment compacts pour comparer rapidement plusieurs dizaines de candidats DOM, tout en capturant des relations sémantiques que TF-IDF ne peut pas représenter. À l'inverse, **all-mpnet-base-v2** améliore légèrement la finesse sémantique, mais son surcoût n'est pas prioritaire dans une exécution de tests où la réponse doit rester interactive. La figure 2.14 illustre cet arbitrage entre qualité attendue et latence opérationnelle.

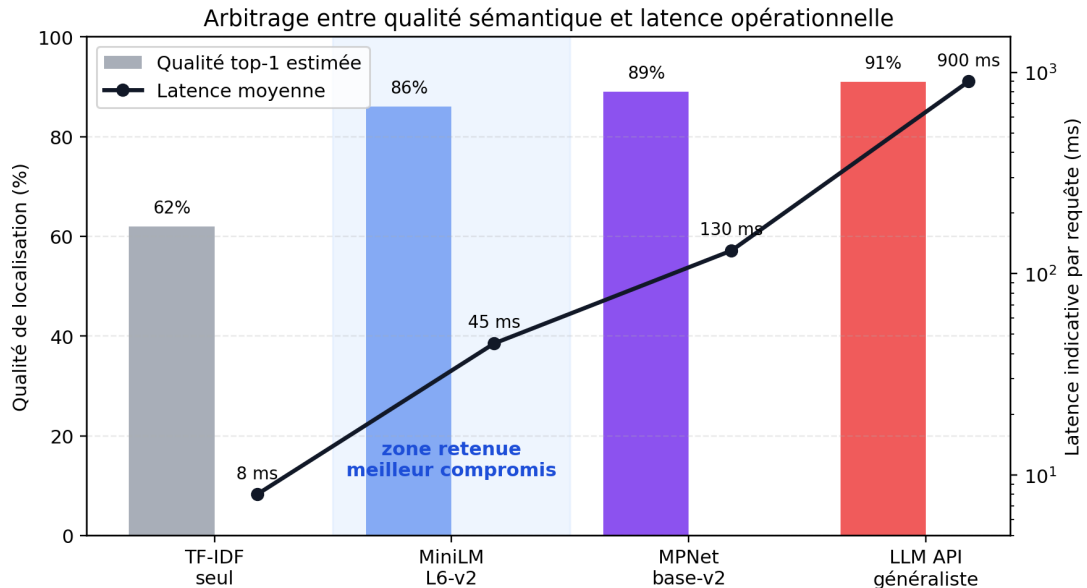


FIGURE 2.14 – Arbitrage entre qualité de localisation et latence des approches candidates

La décision finale a été formalisée sous forme de matrice multicritère. Les critères les plus importants pour ce projet sont la latence CPU, la mémoire, l'intégration Python et la confidentialité des données de test. Sur ces axes, MiniLM obtient un score plus

équilibré que les modèles plus lourds ou les solutions externalisées par API, comme le montre la figure 2.15.

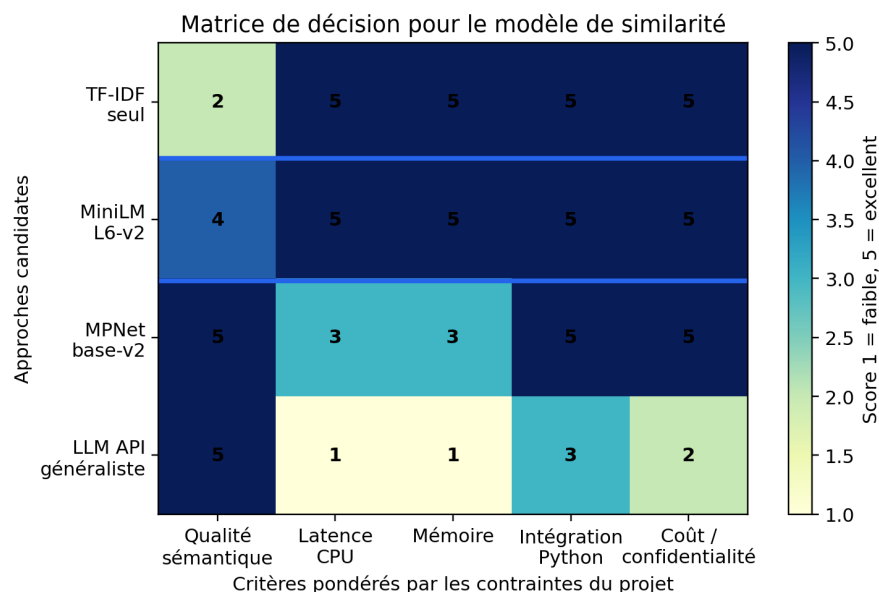


FIGURE 2.15 – Matrice de décision multicritère pour le choix du modèle sémantique

L'évaluation interne menée sur $N = 30$ cas de rupture de locators s'appuie sur le rang du bon candidat (top-1, top-5 et MRR), le score de confiance moyen et le temps d'inférence par requête. Les essais montrent que MiniLM conserve une latence inférieure à la seconde pour l'encodage d'un ensemble de candidats DOM, tout en fournissant un signal suffisamment robuste pour alimenter la décision multi-signeaux avec TF-IDF, SSIM et les règles DOM.

La traduction FR→EN reste optionnelle : elle harmonise les libellés ou sélecteurs rédigés en français avec l'espace d'embedding principalement anglais du modèle MiniLM. Un modèle multilingue a été envisagé, mais la V1 privilégie ce compromis pour conserver une faible latence, limiter l'empreinte mémoire et préserver une architecture facilement reproductible.

2.10 Conclusion

Ce chapitre a comparé les solutions existantes et mis en évidence leurs limites : faible explicabilité, analyse souvent mono-modale et intégration parfois complexe. La solution proposée répond à ces points par un pipeline multi-signeaux, open source, exposé via API REST et accompagné d'un score de confiance.

Chapitre 3

Business Understanding : cadrage métier du self-healing ML

Phase 1 – Business Understanding

Objectif	Cadrer le besoin métier et les critères de succès.
Livrables	Problématique, objectifs, contraintes, critères de succès.
Identité	Palette pastel dédiée à cette étape CRISP-DM afin de distinguer clairement son rôle dans le rapport.

3.1 Introduction

Cette phase transforme le besoin métier en objectifs mesurables. Pour ARCANE, il s'agit de réduire la maintenance des tests E2E en retrouvant automatiquement un élément cible lorsqu'un locator devient invalide.

3.2 Problématique métier

Dans les environnements Agile et DevOps, les interfaces évoluent fréquemment : renommage d'identifiants, ajout de composants, refonte visuelle, changement d'attributs ou réorganisation partielle du DOM. Ces modifications peuvent invalider des tests pourtant fonctionnellement pertinents.

Les impacts métier principaux sont les suivants :

- augmentation du temps de maintenance des tests automatisés ;
- apparition de faux positifs dans les rapports d'exécution ;
- ralentissement des cycles de feedback en CI/CD ;
- perte de confiance progressive dans la suite de tests ;

— risque de désactivation de tests utiles pour réduire le bruit opérationnel.

La problématique retenue est donc la suivante : comment proposer automatiquement un sélecteur de remplacement fiable lorsqu'un locator devient invalide, tout en évitant les réparations incorrectes ?

3.3 Objectifs du projet

L'objectif général est de concevoir un pipeline de *self-healing* piloté par des modèles ML, capable d'assister l'exécution des tests Playwright. Le système doit détecter l'échec d'un sélecteur, analyser la page cible, représenter chaque candidat par des caractéristiques textuelles, structurelles et visuelles, puis retourner un locator exploitable avec un niveau de confiance explicite.

TABLE 3.1 – Objectifs métier et traduction technique

Objectif métier	Traduction technique
Réduire la maintenance manuelle	Détecter les sélecteurs cassés et proposer automatiquement des locators de remplacement.
Préserver la continuité des tests	Relancer l'action ou l'assertion avec le locator corrigé lorsque la confiance ML est suffisante.
Limiter les faux positifs	Refuser le healing lorsque le score composite est faible ou lorsque la cible paraît absente.
Assurer la traçabilité	Persister chaque tentative avec les scores, les candidats, les images et les métadonnées.
Visualiser la performance	Fournir des indicateurs temps réel via une API et un tableau de bord analytique.

3.4 Critères de succès

Les critères de succès sont définis avant la modélisation afin d'éviter une évaluation subjective. Ils couvrent la qualité de décision, la robustesse face aux cas négatifs et le temps de réponse.

TABLE 3.2 – Critères de succès du pipeline ARCANE

Métrique	Objectif	Interprétation
Accuracy	> 95%	Proportion globale de décisions correctes.
Precision	> 90%	Fiabilité des healings acceptés.
Recall	> 95%	Capacité à retrouver les éléments réellement présents.
F1 Score	> 0,92	Compromis entre précision et rappel.
Faux positifs	< 5%	Réparations incorrectes parmi les healings déclarés.
Temps moyen ML	< 60s	Durée acceptable pour le pipeline multi-modal.
Latence API	10–60s	Temps variable selon le nombre de candidats et les modules visuels activés.

3.5 Architecture ML visée

La solution attendue repose sur un enchaînement clair : collecte des entrées, préparation des représentations, scoring par modèles ML, décision contrôlée et restitution d'un résultat exploitable. La figure 3.1 donne une vision synthétique de cette logique.

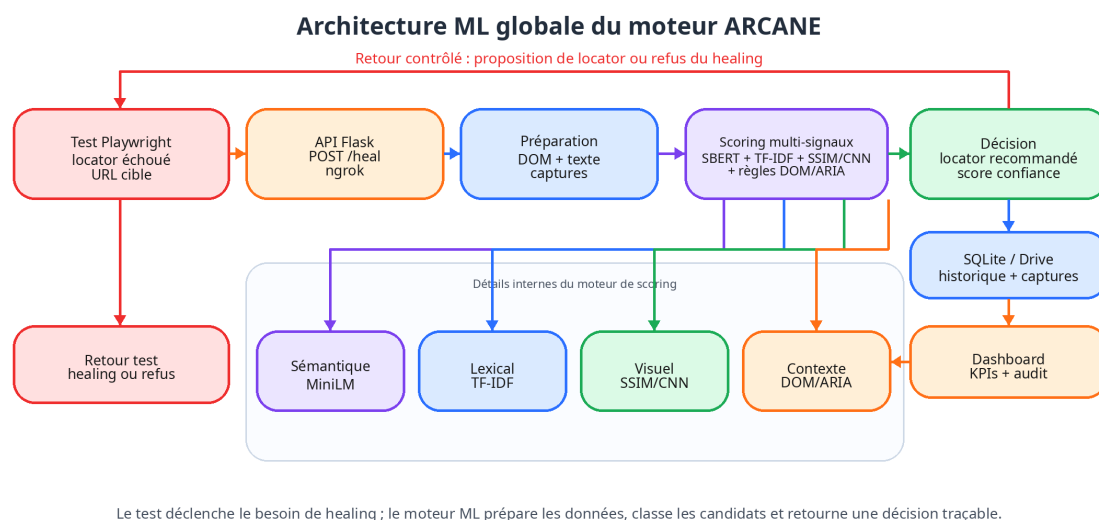


FIGURE 3.1 – Vision globale de l'architecture ML du moteur ARCANE

Cette architecture met en évidence une séparation importante : le test Playwright déclenche le besoin de healing, mais la décision appartient au moteur ML. Celui-ci combine les informations issues du DOM, du texte et des captures afin de produire un classement de candidats plutôt qu'une réponse binaire immédiate.

3.6 Contraintes et hypothèses

Le projet est contraint par l'environnement d'exécution choisi. Le moteur doit pouvoir fonctionner dans Google Colab, avec un GPU optionnel et des ressources limitées. L'exposition de l'API repose sur ngrok, ce qui correspond à un contexte de prototype avancé plutôt qu'à une infrastructure de production permanente.

Les hypothèses structurantes sont les suivantes :

- la page cible conserve une structure DOM partiellement stable ;
- les sélecteurs cassés conservent des indices exploitables dans leur forme, leur contexte ou leur texte ;
- les attributs sémantiques, le texte visible, la structure DOM et l'apparence visuelle restent suffisamment informatifs ;
- les cas sans cible doivent être explicitement rejetés afin de limiter les faux positifs.

3.7 Planification CRISP-DM

La figure 3.2 synthétise la planification retenue. Le cadrage prévoyait une période allant du 2 février au 31 juillet 2026 ; la finalisation effective du rapport et du prototype a été réalisée le 4 juillet 2026. La phase de modélisation concentre l'effort principal, car elle regroupe l'encodage sémantique, le scoring multi-modal, la fusion des scores et la décision finale.

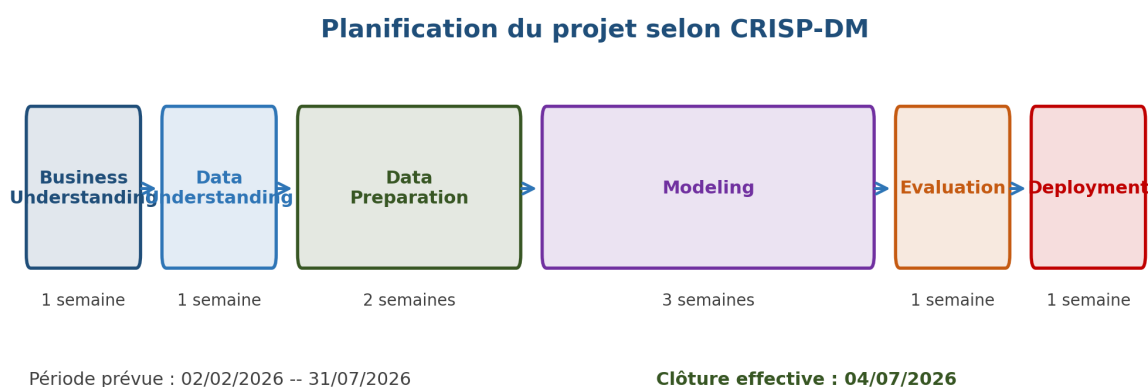


FIGURE 3.2 – Planification du projet ARCANE selon les six phases CRISP-DM

3.8 Conclusion

Cette phase relie le besoin métier à la solution ML proposée. Elle fixe les critères de réussite, notamment la réduction de maintenance, la traçabilité des décisions et la limitation des réparations abusives.

Chapitre 4

Data Understanding : compréhension des données du projet

Phase 2 – Data Understanding

Objectif	Comprendre les données disponibles et leurs limites.
Livrables	Page Arcane Shop, DOM extrait, scénarios, vocabulaire UI.
Identité	Palette pastel dédiée à cette étape CRISP-DM afin de distinguer clairement son rôle dans le rapport.

4.1 Introduction

Cette phase identifie les sources de données, leur structure et leurs limites. AR-CANE s'appuie sur une application web instrumentée, un DOM extrait dynamiquement et des scénarios simulant des sélecteurs cassés.

4.2 Sources de données

Le projet ARCANE repose sur trois sources principales : la page web de test, les éléments extraits du DOM et les scénarios d'évaluation. La figure 4.1 présente cette organisation.

Sources de données exploitées

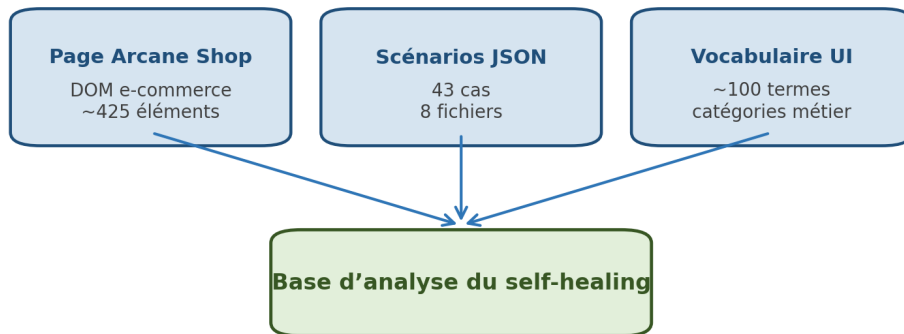


FIGURE 4.1 – Sources de données exploitées par le pipeline de self-healing

4.2.1 Page web cible

La page de référence est une boutique e-commerce fictive appelée *Arcane Shop*. Elle offre un contexte fonctionnel suffisamment riche pour représenter une interface web moderne : navigation, catalogue produit, modales, panier, wishlist, recherche, filtres et tunnel de checkout.

- navigation principale avec onglets produits ;
- section hero avec indicateurs commerciaux ;
- grille de 24 fiches produits ;
- modale produit avec onglets ;
- panier latéral, wishlist et modale de recherche ;
- checkout multi-étapes : informations, livraison, paiement, confirmation.

Cette diversité permet de tester le healing sur des éléments de nature différente : boutons, liens, champs de formulaire, badges, sections, panneaux latéraux et composants modaux.

4.2.2 DOM extrait par Playwright

L'extraction s'effectue avec Playwright à partir d'une sélection d'éléments interactifs et sémantiques. En moyenne, environ 425 éléments sont collectés sur la page cible. Chaque élément est représenté par un dictionnaire normalisé.

TABLE 4.1 – Principaux attributs extraits du DOM

Attribut	Rôle	Exemple
id	Identifiant unique ou semi-stable.	promo-bar
tag	Type de balise HTML.	button, div
class	Classes CSS associées.	product-card
text	Texte visible par l'utilisateur.	Add to Cart
data-testid	Attribut dédié aux tests.	tab-featured
aria-label	Label d'accessibilité.	Search
placeholder	Indication de saisie.	Search products

4.2.3 Scénarios d'évaluation

Les scénarios sont stockés dans huit fichiers JSON et totalisent 43 cas. Chaque cas contient un sélecteur cassé, un texte de référence, un seuil de confiance, un indicateur `has_target` et l'élément attendu. Ces scénarios constituent la base expérimentale du projet : ils permettent d'observer comment le moteur ML classe les candidats et comment il distingue un élément réellement retrouvable d'un cas sans cible valide.

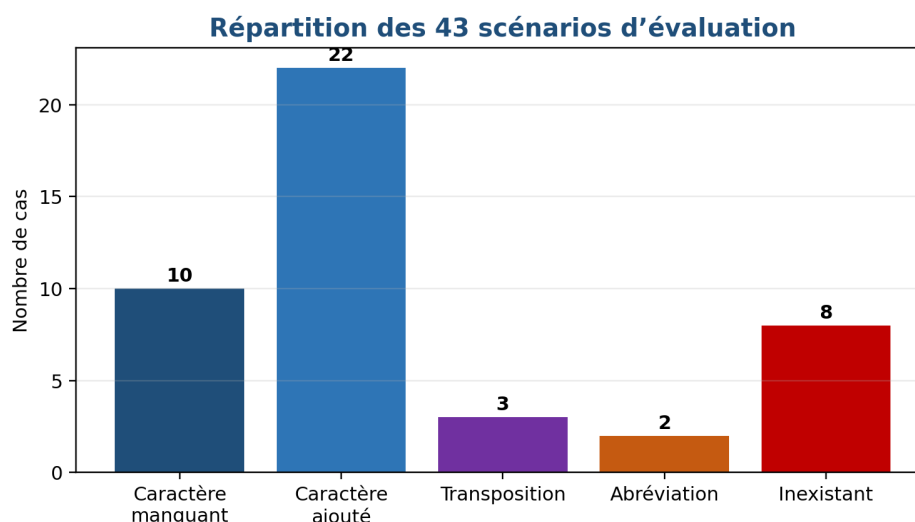


FIGURE 4.2 – Répartition des scénarios utilisés pour tester le moteur ML

4.3 Analyse du vocabulaire UI

L'analyse du DOM a permis d'identifier un vocabulaire d'environ 100 termes récurrents. Ce vocabulaire est indispensable pour interpréter les sélecteurs et enrichir les représentations textuelles avant encodage.

— **Navigation** : nav, menu, tab, breadcrumb, pagination ;

- **Formulaire** : input, label, select, checkbox, submit ;
- **E-commerce** : cart, wishlist, checkout, product, price ;
- **Modal/UX** : modal, overlay, tooltip, toast, drawer ;
- **Action** : search, save, cancel, close, open.

4.4 Qualité et limites des données

Les données couvrent efficacement les variations de sélecteurs, les erreurs d'attributs et les cas négatifs. Elles restent cependant limitées à une seule page de test et à des sélecteurs majoritairement en *kebab-case*. Les changements structurels majeurs, les refontes complètes et les conventions *camelCase* ou *snake_case* ne sont pas représentés de manière exhaustive.

Cette limite justifie l'utilisation d'une approche ML multi-signaux plutôt qu'une règle unique dépendante d'un format de sélecteur. Le modèle doit exploiter simultanément le sens des libellés, les attributs DOM, les informations d'accessibilité et les captures visuelles.

4.5 Conclusion

Cette phase confirme que les données disponibles permettent d'évaluer une première version du pipeline. Elles couvrent la réparation automatique, le refus de healing et les principales limites à considérer.

Chapitre 5

Data Preparation : préparation des données pour le pipeline ML

Phase 3 – Data Preparation

Objectif	Transformer les entrées brutes en représentations ML.
Livrables	Parsing, filtrage, features DOM, texte, images et embeddings.
Identité	Palette pastel dédiée à cette étape CRISP-DM afin de distinguer clairement son rôle dans le rapport.

5.1 Introduction

Cette phase transforme les entrées brutes — sélecteur cassé, DOM, texte et captures — en représentations exploitables. La qualité de ces données conditionne directement la fiabilité du scoring.

Rôle de la préparation des données dans ARCANE

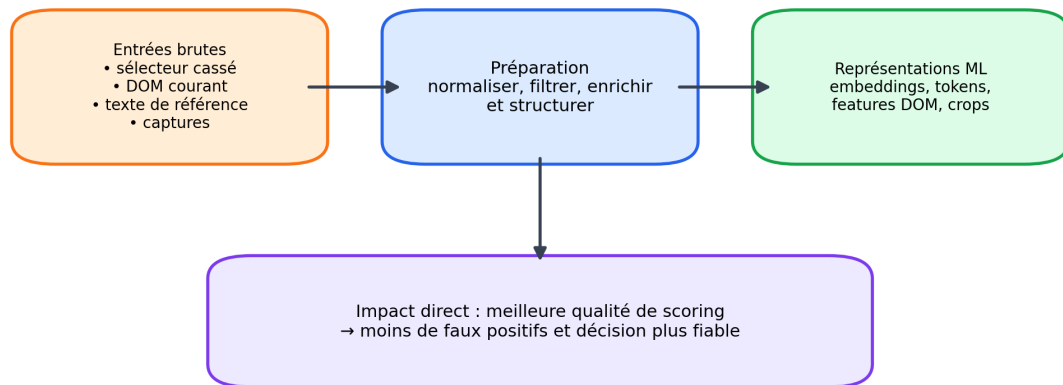


FIGURE 5.1 – Rôle de la préparation des données dans la chaîne de décision ARCANE

5.2 Architecture de préparation

La figure 5.2 présente le pipeline de préparation retenu. Il convertit progressivement un sélecteur cassé en vecteur sémantique et en données auxiliaires utilisables par les étages textuels et visuels.

Pipeline de préparation des données

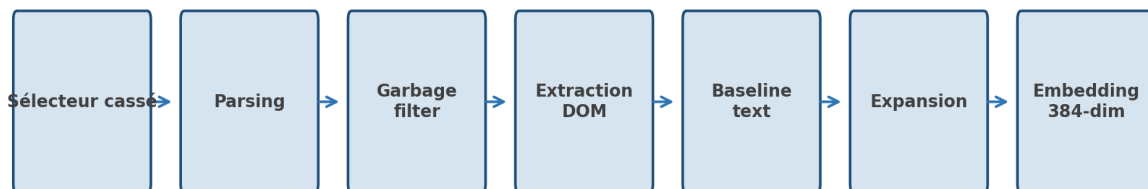


FIGURE 5.2 – Pipeline de préparation des données avant modélisation

5.3 Parsing du sélecteur

La fonction de parsing décompose le sélecteur cassé afin d'en extraire les informations exploitables : identifiant, classe, attribut `data-testid`, type d'input ou texte potentiel. Le résultat est utilisé comme une source de caractéristiques pour les modèles ML et pour la comparaison multi-signaux.

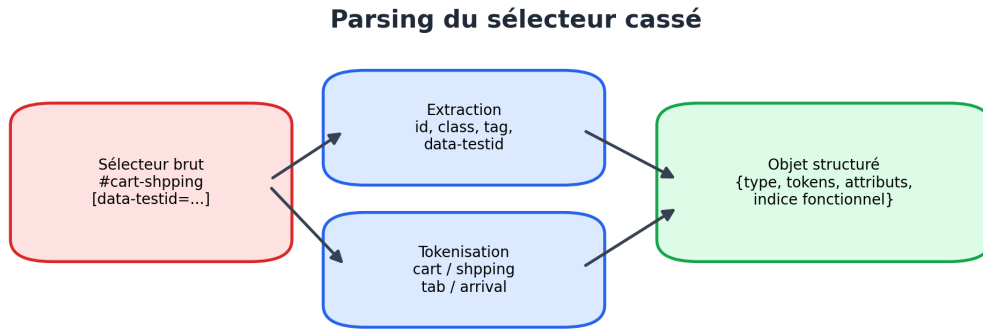
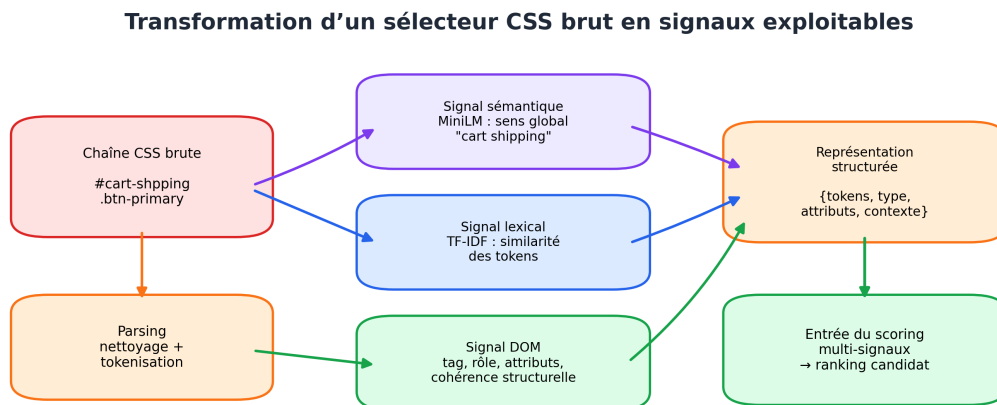


FIGURE 5.3 – Décomposition d’un sélecteur cassé en informations exploitables

Cette étape permet de passer d’une chaîne CSS brute à une représentation structurée. Les tokens extraits alimentent ensuite l’encodage sémantique, la similarité lexicale et les règles de cohérence DOM.



Le sélecteur n’est plus traité comme une simple chaîne : il devient une source de signaux complémentaires pour la décision ML.

FIGURE 5.4 – Passage d’une chaîne CSS brute vers des signaux sémantiques, lexicaux et DOM

5.4 Feature engineering multi-modal

Chaque élément candidat est transformé en vecteur de caractéristiques. Ce vecteur regroupe des informations structurelles, textuelles, sémantiques et visuelles. La figure 5.5 illustre cette construction.

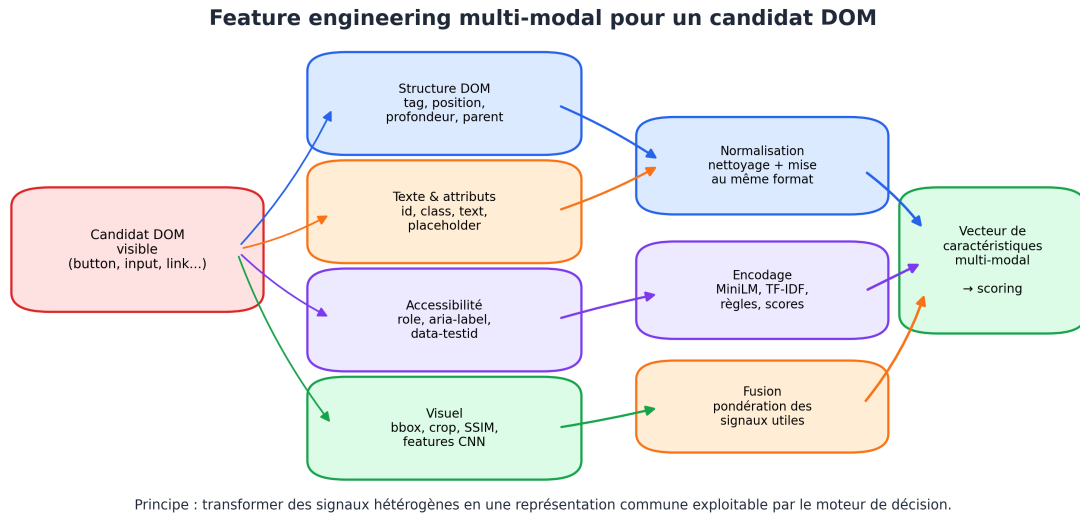


FIGURE 5.5 – Construction des caractéristiques utilisées par les modèles ML

L'intérêt de cette étape est de rendre comparables des éléments hétérogènes. Un bouton, un champ de saisie ou une section peuvent ainsi être évalués dans un même espace de décision, même si leurs attributs HTML ne sont pas identiques.

La figure 5.5 illustre cette normalisation en regroupant les attributs DOM, les textes visibles, les indices d'accessibilité et les informations visuelles dans une représentation commune.

5.5 Détection des sélecteurs manifestement invalides

Un filtre précoce identifie les sélecteurs de type *garbage*. Ces cas contiennent généralement des marqueurs artificiels tels que `xyz`, `abc`, `dummy`, `fake`, `invalid` ou des suffixes numériques suspects comme `999` et `123`. L'objectif est d'éviter de lancer un pipeline coûteux sur des cas où aucun élément réel n'est attendu.

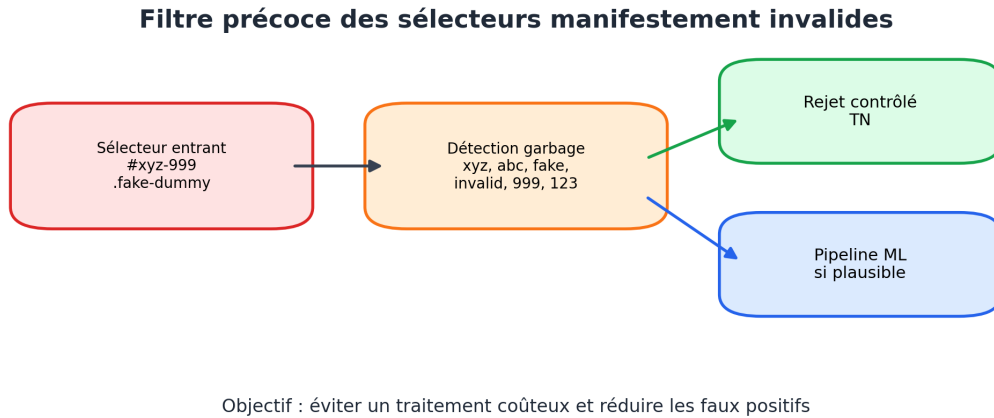


FIGURE 5.6 – Filtre précoce des sélecteurs manifestement invalides

Ce mécanisme constitue un garde-fou important contre les faux positifs. Dans le jeu de test, les huit cas inexistantes sont conçus pour évaluer précisément cette capacité de refus.

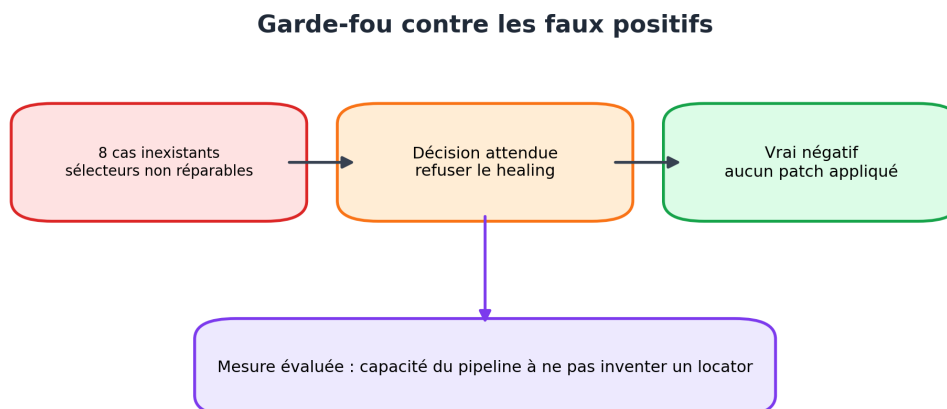


FIGURE 5.7 – Garde-fou expérimental contre les faux positifs de healing

5.6 Extraction et filtrage du DOM

L'extraction DOM est réalisée par Playwright. Les éléments invisibles ou non pertinents sont exclus : champs `hidden`, dimensions nulles, styles `display:none`, `visibility:hidden` ou éléments sans rôle exploitable.

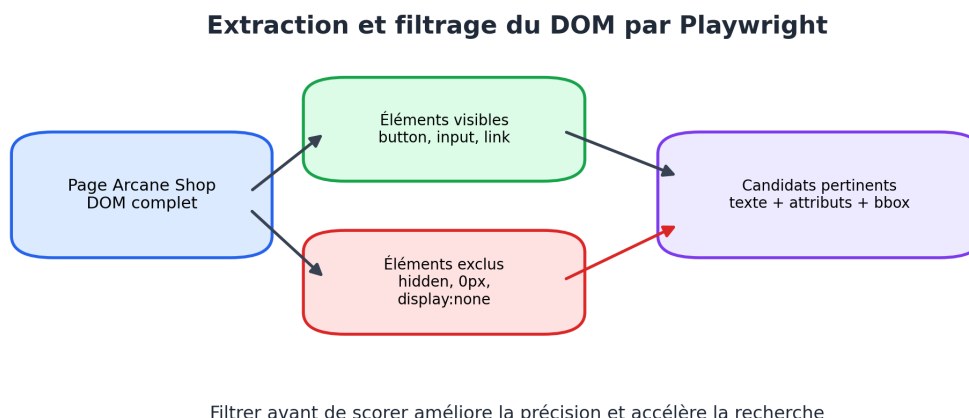


FIGURE 5.8 – Extraction DOM et filtrage des candidats non pertinents

Pour chaque candidat visible, le pipeline construit une représentation textuelle à partir de ses attributs : `id`, `class`, `text`, `data-testid`, `aria-label`, `name` et `placeholder`. Cette représentation sert ensuite aux modèles de similarité.

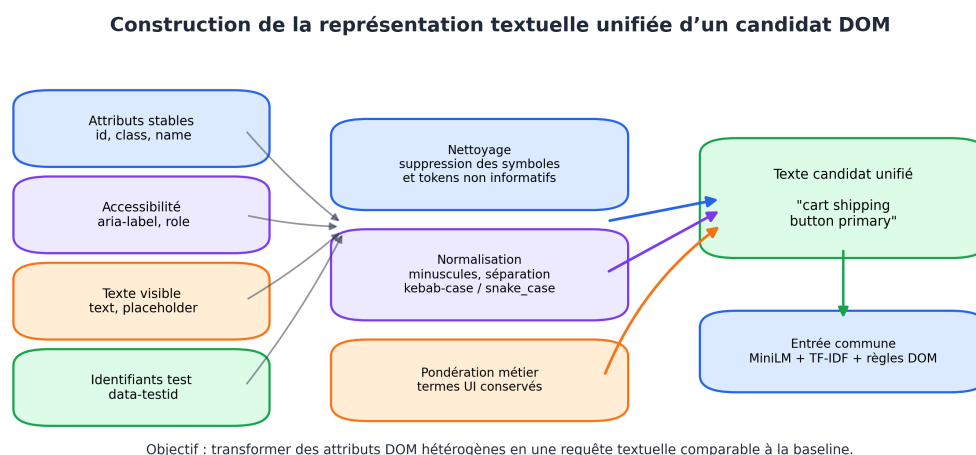


FIGURE 5.9 – Construction d'une représentation textuelle unifiée pour chaque candidat DOM

5.7 Construction du texte de référence

La fonction `selector_to_baseline_text` transforme le sélecteur cassé en requête textuelle. Elle supprime les symboles CSS non informatifs, segmente les mots en *kebab-case*, normalise les tokens et privilégie les termes porteurs de sens.

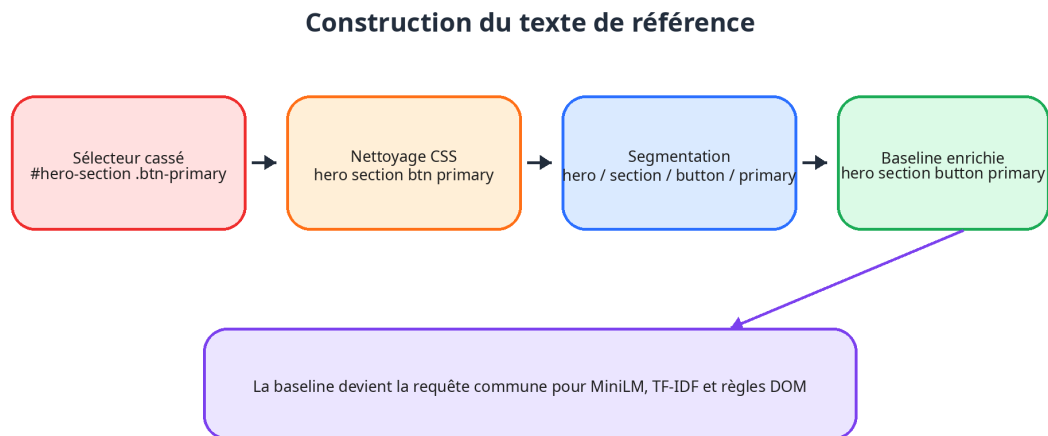


FIGURE 5.10 – Construction progressive d’une baseline textuelle à partir du sélecteur cassé

TABLE 5.1 – Exemples de transformation vers une baseline textuelle

Sélecteur cassé	Baseline textuelle produite
#promo-barr	promo barr enrichi par le contexte de bannière promotionnelle
[data-testid="tab-new-arrival"]	tab new arrival rapproché des onglets de navigation
#cart-shpping	cart shpping comparé aux signaux panier et livraison
#hero-section .btn-primary	hero section button primary

5.8 Enrichissement lexical

L’enrichissement lexical améliore la qualité des représentations textuelles avant l’encodage ML. Les tokens courts ou ambigus sont rapprochés du vocabulaire UI, tandis que certains termes techniques sont préservés afin de conserver leur sens fonctionnel : `btn`, `nav`, `modal`, `tab`, `row`, `col`, `search`, `filter`.

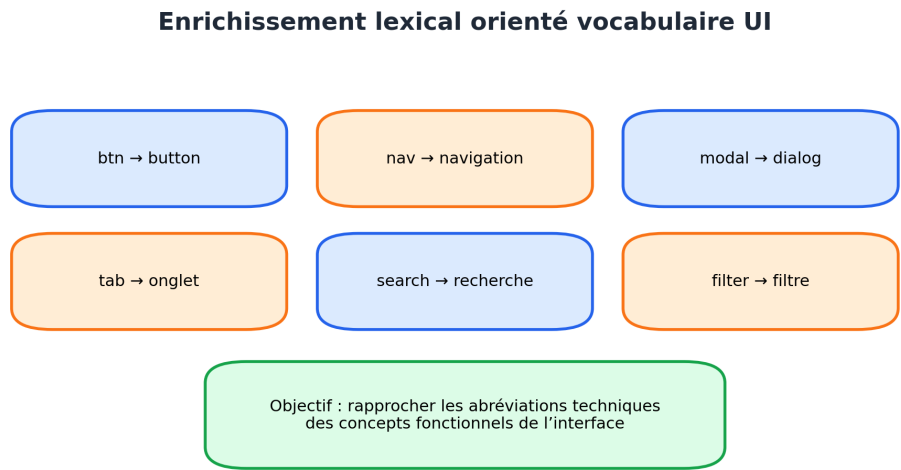


FIGURE 5.11 – Enrichissement lexical à partir du vocabulaire fonctionnel des interfaces web

5.9 Préparation des représentations sémantiques et visuelles

Le texte préparé est encodé avec `all-MiniLM-L6-v2`, produisant un vecteur de 384 dimensions. Pour les étapes visuelles, le pipeline capture également une image baseline et des *crops* candidats. Ces images alimentent SSIM et le modèle CNN lorsque l'analyse visuelle est nécessaire.

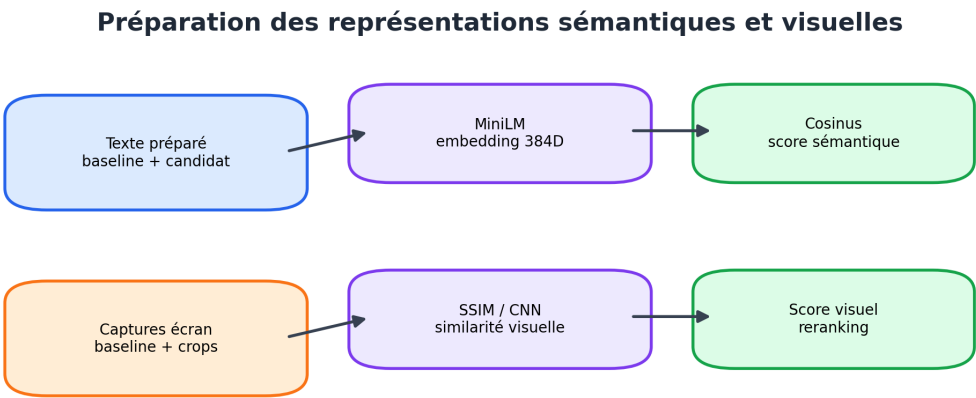


FIGURE 5.12 – Préparation conjointe des représentations sémantiques et visuelles

5.10 Conclusion

La préparation des données fait le lien entre l'échec Playwright et le scoring ML. Elle produit des indices textuels, DOM et visuels nécessaires à une décision de healing fiable.

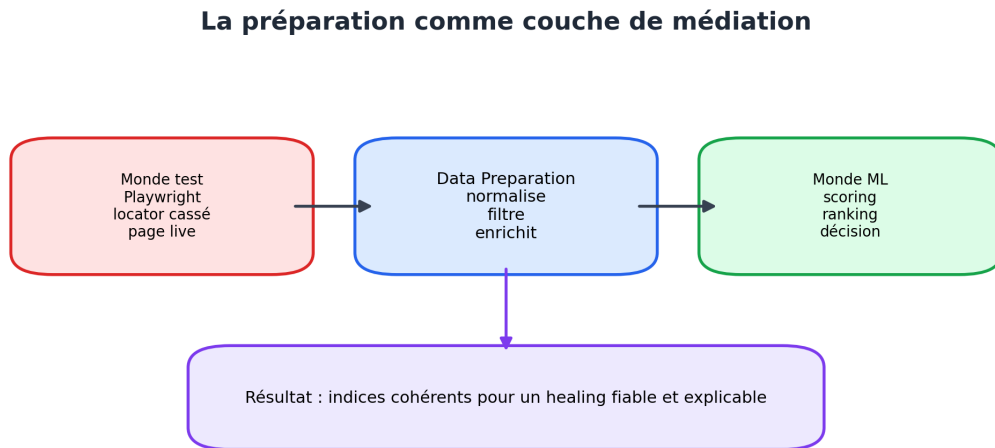


FIGURE 5.13 – La préparation des données comme couche de médiation entre Playwright et le scoring ML

Chapitre 6

Modeling : conception du moteur ML de self-healing

Phase 4 – Modeling

Objectif	Concevoir le moteur de scoring multi-modal.
Livrables	MiniLM, TF-IDF, contexte DOM, SSIM, CNN et fusion des scores.
Identité	Palette pastel dédiée à cette étape CRISP-DM afin de distinguer clairement son rôle dans le rapport.

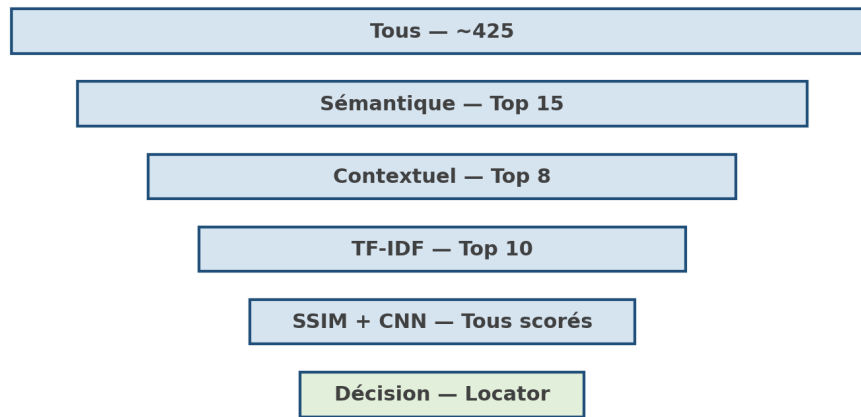
6.1 Introduction

La modélisation constitue le cœur technique du projet. Elle classe les candidats DOM à partir de signaux sémantiques, lexicaux, contextuels et visuels afin de retrouver l'élément cible.

6.2 Vue d'ensemble du pipeline de scoring

Le pipeline ARCANE fonctionne comme un entonnoir de candidats. Les éléments extraits du DOM sont progressivement filtrés, scorés et réordonnés jusqu'à produire un locator recommandé. La figure 6.1 résume cette logique.

Entonnoir de scoring multi-étages



Fusion pondérée : contexte 0,35 · TF-IDF 0,25 · SSIM 0,15 · CNN 0,15

FIGURE 6.1 – Entonnoir de scoring multi-étages du moteur ARCANE

6.3 Modèle de décision ML

Le moteur ne retourne pas simplement le premier élément ressemblant au sélecteur cassé. Il calcule un score composite pour chaque candidat, puis compare ce score à un seuil de confiance. La figure 6.2 présente cette logique de décision.

Règles de décision fondées sur le score ML

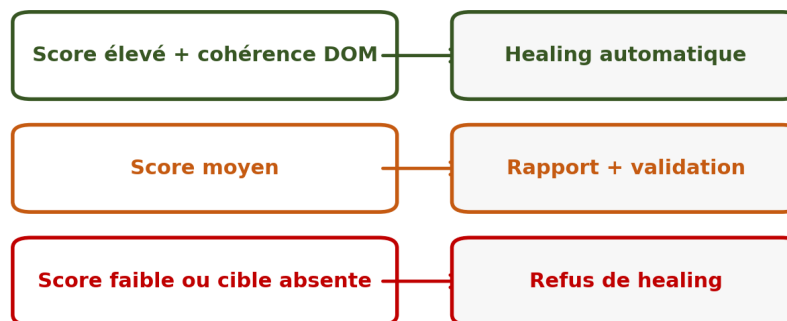


FIGURE 6.2 – Règles de décision fondées sur le score ML composite

Cette stratégie permet de distinguer trois situations : healing automatique lorsque les signaux convergent, validation humaine lorsque le score est intermédiaire, et refus lorsque la cible semble absente ou insuffisamment fiable.

6.4 Fonction de fusion des scores

Chaque étage produit un score partiel. Ces scores sont fusionnés progressivement afin d'éviter qu'un seul signal domine systématiquement la décision. La fusion suit une logique pondérée :

$$S_{final} = f(S_{sem}, S_{ctx}, S_{tfidf}, S_{ssim}, S_{cnn}) \quad (6.1)$$

Les poids documentés dans le projet sont les suivants : contexte 0,35, TF-IDF 0,25, SSIM 0,15 et CNN 0,15. Le score sémantique constitue le point de départ du classement, puis les autres signaux ajustent la confiance.

6.5 Stage 1 : similarité sémantique

Le premier étage utilise le modèle `all-MiniLM-L6-v2`. Chaque élément candidat est représenté par une phrase synthétique issue de ses attributs DOM. La requête issue du sélecteur cassé est encodée dans le même espace vectoriel. La similarité cosinus permet ensuite de classer les candidats.

Cet étage retient les quinze meilleurs éléments. Il est utile lorsque le sélecteur cassé conserve un sens proche de l'élément attendu, même si la forme exacte diffère.

6.6 Stage 2 : scoring contextuel

Le scoring contextuel exploite les attributs DOM stables : `id`, `data-testid`, `aria-label`, `name`, `placeholder`, rôle et tag HTML. Les correspondances exactes ou partielles sont valorisées lorsque les tokens du sélecteur cassé se retrouvent dans les attributs du candidat.

Cet étage permet de réduire la dépendance au texte visible et de privilégier les signaux conçus pour les tests automatisés ou l'accessibilité.

6.7 Stage 3 : similarité textuelle TF-IDF

L'étage TF-IDF évalue la proximité lexicale entre la baseline textuelle et les descriptions candidates. Contrairement aux embeddings, TF-IDF valorise les mots rares et discriminants. Il est donc pertinent pour distinguer des éléments proches mais portant des termes différents, par exemple `cart`, `wishlist`, `checkout` ou `search`.

6.8 Stage 4 : similarité visuelle SSIM

SSIM compare la structure visuelle entre une image baseline et les crops candidats. Il complète les signaux textuels lorsque l'apparence de l'élément reste stable malgré une modification du DOM. Le score SSIM est particulièrement utile pour des boutons, panneaux ou cartes produit dont la position et la forme restent proches.

6.9 Stage 5 : vision IA par CNN

Le modèle EfficientNet-B0, pré-entraîné sur ImageNet, est utilisé pour extraire une représentation visuelle plus abstraite. La similarité cosinus entre les vecteurs image permet d'évaluer la proximité perceptuelle entre la baseline et les candidats. Cet étage est plus coûteux que SSIM et s'inscrit donc dans une logique de complément pour les cas complexes.

6.10 Génération des locators

Une fois le candidat retenu, le système génère plusieurs locators ordonnés par stabilité : `data-testid`, rôle accessible, label, placeholder, texte visible, identifiant CSS, classe ou sélecteur composite. Cette hiérarchie favorise les locators robustes et lisibles par les ingénieurs QA.

6.11 Conclusion

La modélisation combine plusieurs signaux pour produire une décision précise et traçable. Les scores intermédiaires facilitent l'audit et expliquent le choix du locator recommandé.

Chapitre 7

Evaluation : validation du pipeline ML de self-healing

Phase 5 – Evaluation

Objectif	Mesurer la performance et analyser les erreurs.
Livrables	Matrice de confusion, accuracy, precision, recall, F1 et latence.
Identité	Palette pastel dédiée à cette étape CRISP-DM afin de distinguer clairement son rôle dans le rapport.

7.1 Introduction

L'évaluation vérifie si le pipeline respecte les objectifs fixés. Elle analyse la précision, les faux positifs, les faux négatifs, les causes d'erreur et la latence du moteur.

7.2 Protocole d'évaluation

Le protocole s'appuie sur les 43 scénarios JSON. Pour chaque cas, le système reçoit le sélecteur cassé et le texte de référence, appelle l'endpoint `/heal`, puis compare la réponse obtenue avec l'élément attendu.

1. charger les scénarios d'évaluation ;
2. appeler l'API avec `broken_selector` et `baseline_text` ;
3. récupérer le locator recommandé et le score de confiance ;
4. vérifier si l'élément retourné correspond à la vérité terrain ;
5. calculer accuracy, precision, recall, F1, FPR et FNR.

7.3 Matrice de confusion

Les cas positifs correspondent aux sélecteurs réparables. Les cas négatifs correspondent aux sélecteurs inexistant, pour lesquels le bon comportement est de refuser le healing.

TABLE 7.1 – Matrice de confusion attendue après ajustement du pipeline ML

	Réel positif	Réel négatif	Total
Prédit positif	TP = 34	FP = 1	35
Prédit négatif	FN = 0	TN = 8	8
Total	34	9	43

7.4 Résultats et évolution des métriques

Les premiers résultats ont mis en évidence une performance insuffisante, principalement due à des faux positifs. Après ajustement de la stratégie de décision et des seuils de confiance, les métriques attendues progressent fortement, comme le montre la figure 7.1.

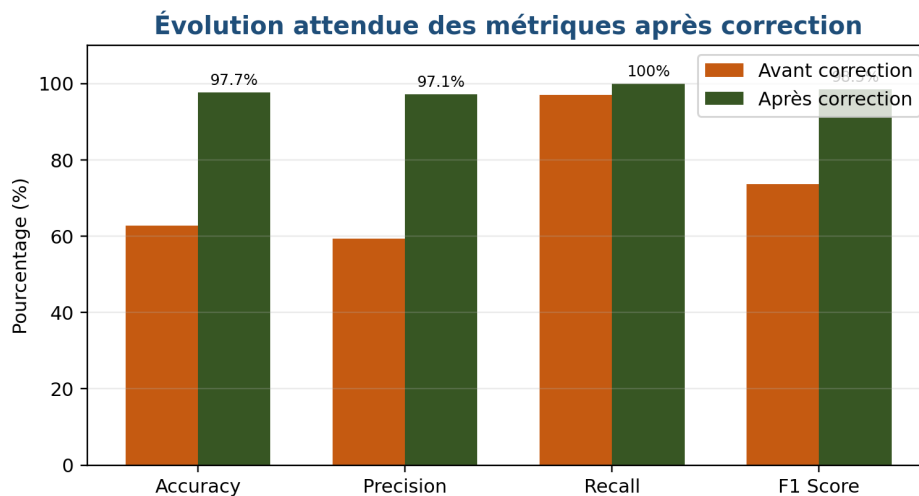


FIGURE 7.1 – Comparaison des métriques avant et après ajustement du pipeline ML

TABLE 7.2 – Synthèse des métriques avant et après ajustement

Métrique	Avant	Après	Gain
Accuracy	62,8%	97,7%	+34,9 pts
Precision	59,3%	97,1%	+37,8 pts
Recall	97,0%	100%	+3,0 pts
F1 Score	73,6%	98,5%	+24,9 pts
Faux positifs	15	1	-14
Faux négatifs	1	0	-1

7.5 Analyse des causes racines

L’analyse des erreurs montre que la performance dépend fortement de la qualité du parsing et de la stratégie d’agrégation. Une expression régulière trop stricte peut empêcher l’extraction correcte d’un attribut `data-testid`. De même, un seuil de confiance trop faible peut accepter un candidat visuellement ou lexicalement proche mais fonctionnellement différent.

Les ajustements retenus portent donc sur trois leviers : amélioration du parsing, renforcement du seuil de confiance et meilleure pondération des signaux sémantiques par rapport aux signaux visuels. L’objectif est de favoriser les candidats qui sont cohérents à la fois dans le texte, le DOM et l’apparence.

7.6 Performances temporelles

Les temps de réponse varient selon le nombre de candidats et les modules ML activés. La détection d’un cas manifestement invalide est rapide, tandis que le pipeline complet mobilise les embeddings, TF-IDF, SSIM et CNN.

TABLE 7.3 – Temps de réponse observés selon la stratégie ML

Stratégie	Temps moyen	Cas typiques
Filtrage initial	< 1s	Sélecteurs manifestement inexistant.
Scoring sémantique et textuel	3–10s	Candidats discriminés par le DOM et le texte.
Pipeline ML complet	30–90s	Cas ambigus nécessitant les signaux visuels.

7.7 Conclusion

L'évaluation montre que la performance dépend des modèles, mais aussi des seuils et des pondérations. Ces réglages réduisent les faux positifs et rapprochent le système des objectifs métier.

Chapitre 8

Deployment : intégration et mise à disposition du système ML

Phase 6 – Deployment

Objectif	Rendre le système exploitable par les tests et le dashboard.
Livrables	API Flask, ngrok, Playwright, SQLite et supervision Angular.
Identité	Palette pastel dédiée à cette étape CRISP-DM afin de distinguer clairement son rôle dans le rapport.

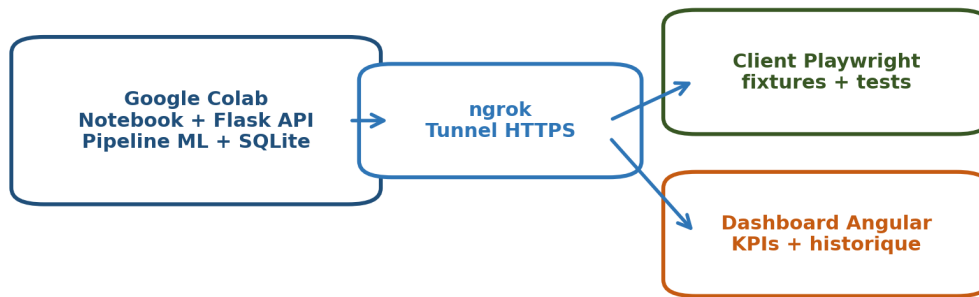
8.1 Introduction

Cette phase rend le pipeline exploitable sous forme de prototype. Le déploiement combine un notebook Colab, une API Flask exposée par ngrok, un client Playwright et un dashboard Angular.

8.2 Architecture de déploiement

La figure 8.1 synthétise les composants du déploiement ARCANE. Le notebook exécute le moteur ML, l'API reçoit les requêtes, le client Playwright intercepte les échecs et le dashboard visualise les résultats.

Architecture de déploiement ARCANE



Endpoints : POST /heal · GET /dashboard-data · GET /analytics · GET /history · GET /ping

FIGURE 8.1 – Architecture de déploiement du système ARCANE

8.3 Séquence d'intégration

La figure 8.2 illustre le dialogue entre le test Playwright, l'API Flask, le moteur ML et la persistance. Cette séquence montre que le healing est traité comme un service externe : le test signale l'échec, le moteur analyse, puis l'API retourne un résultat structuré.

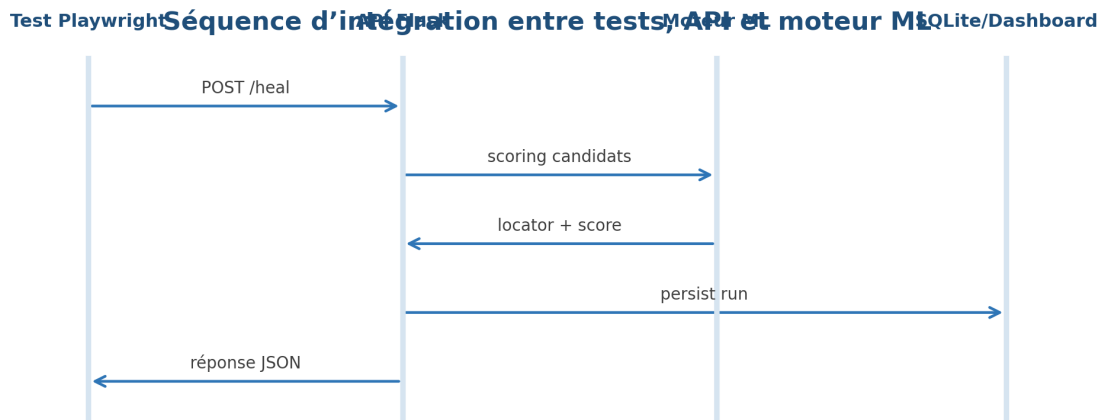


FIGURE 8.2 – Séquence d'intégration entre Playwright, Flask et le moteur ML

8.4 Composant moteur : notebook Colab

Le notebook Colab constitue le cœur de calcul. Il charge les modèles, exécute le pipeline de scoring, produit les locators et persiste l'historique. L'environnement recommandé est un runtime GPU T4 pour accélérer EfficientNet-B0, mais l'exécution CPU reste possible.

Les éléments persistants sont stockés dans Google Drive : base SQLite, captures baseline et crops de candidats. Cette persistance permet de conserver l'historique malgré la nature temporaire des sessions Colab.

8.5 API Flask et endpoints

L'API Flask expose les fonctionnalités du moteur au reste du système. Le point d'entrée principal est `POST /heal`, qui reçoit un sélecteur cassé et retourne le locator recommandé, les alternatives, les scores et les métadonnées.

TABLE 8.1 – Principaux endpoints exposés par l'API Flask

Endpoint	Méthode	Rôle
<code>/heal</code>	POST	Réparer un sélecteur cassé.
<code>/dashboard-data</code>	GET	Fournir les données temps réel du dashboard.
<code>/analytics</code>	GET	Calculer les statistiques globales.
<code>/history</code>	GET	Lister les tentatives de healing.
<code>/screenshot/ annotated</code>	GET	Retourner la capture annotée de la dernière exécution.
<code>/ping</code>	GET	Vérifier l'état du service.

8.6 Client Playwright

Le projet Playwright consomme l'API et intègre le self-healing dans les tests. L'interception repose sur trois couches de proxy JavaScript :

- **Page proxy** : attache les métadonnées du sélecteur au locator ;
- **Locator proxy** : intercepte les actions telles que `click`, `fill`, `hover` ou `check` ;
- **Assertion proxy** : rejoue certaines assertions après healing lorsque cela est pertinent.

Le client supporte plusieurs formats de locators : CSS bruts, `page.locator`, `getByTestId`, `getByRole`, `getByText`, `getByLabel` et `getByPlaceholder`.

8.7 Dashboard Angular

Le dashboard ARCANE est une application Angular destinée à visualiser les performances du pipeline. Il affiche les KPIs, l'historique des runs, les scores par étage, les candidats classés, les captures et les tendances temporelles.

Vue fonctionnelle du tableau de bord

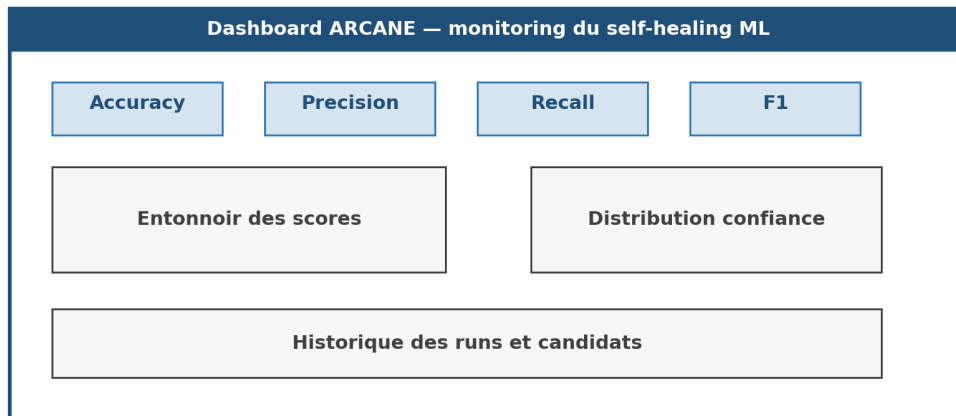


FIGURE 8.3 – Vue fonctionnelle du tableau de bord de supervision

Les intervalles de polling sont différenciés : deux secondes pour le dashboard, quatre secondes pour l'historique et six secondes pour les analytiques. Cette stratégie permet une visualisation quasi temps réel sans surcharger l'API.

8.8 Sécurité et bonnes pratiques

Le prototype applique plusieurs règles de sécurité adaptées à son contexte : stockage du `NGROK_TOKEN` dans les secrets Colab, restriction CORS au dashboard local, sauvegarde des fichiers de test avant patching et timeout long pour les requêtes ML. Ces choix restent compatibles avec un prototype, mais une industrialisation nécessiterait un serveur permanent, une authentification, une journalisation centralisée et un reverse proxy contrôlé.

8.9 Conclusion

Le déploiement rend le pipeline utilisable par Playwright et observable via un dashboard. L'architecture reste légère, mais prépare les interfaces nécessaires à une industrialisation future.

Conclusion générale

Ce rapport a présenté ARCANE, un système de *self-healing* destiné à réduire la fragilité des tests E2E face aux évolutions du DOM. L'approche combine des signaux sémantiques, lexicaux, structurels et visuels pour retrouver l'élément le plus pertinent.

La démarche CRISP-DM a structuré le projet depuis le cadrage métier jusqu'au déploiement du prototype via Flask, Playwright et un dashboard de supervision. Les scores produits rendent la décision traçable et facilitent l'audit des réparations proposées.

Ce travail constitue une base solide pour diminuer la maintenance des tests automatisés. Les perspectives portent principalement sur l'évaluation à plus grande échelle, l'ajustement automatique des pondérations et l'industrialisation du déploiement.

Bibliographie

- [1] Microsoft, “Playwright documentation,” 2024. Consulté le 02/04/2026 ; utilisé pour les tests E2E et les locators DOM.
- [2] N. Reimers and I. Gurevych, “Sentence-bert : Sentence embeddings using siamese bert-networks,” *Proceedings of EMNLP*, 2019. Consulté le 03/03/2026 ; utilisé pour l’encodage sémantique via embeddings.
- [3] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, 1988. Consulté le 10/02/2026 ; utilisé pour expliquer la pondération TF-IDF et la similarité textuelle.
- [4] Scikit-learn Developers, “Scikit-learn machine learning library,” 2024. Consulté le 18/02/2026 ; utilisé pour la vectorisation TF-IDF, la similarité cosinus et les expérimentations ML.
- [5] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, “Image quality assessment : From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, 2004. Consulté le 15/02/2026 ; utilisé pour la métrique SSIM dans la similarité visuelle.
- [6] OpenCV Team, “Opencv library,” 2024. Consulté le 20/02/2026 ; utilisé pour le traitement d’image et les crops candidats.
- [7] J. Tiedemann and S. Thottingal, “Opus-mt – building open translation services for the world,” in *Proceedings of the EAMT*, 2020. Consulté le 08/03/2026 ; utilisé pour la traduction automatique FR–EN.
- [8] Pallets, “Flask web framework,” 2024. Consulté le 10/04/2026 ; utilisé pour exposer l’API REST du prototype.
- [9] Ngrok Inc., “Ngrok documentation,” 2024. Consulté le 12/04/2026 ; utilisé pour l’exposition HTTPS du notebook.
- [10] Microsoft Developer, “Playwright tutorials and conference videos.” YouTube videos, 2025. Consulté le 05/06/2026 ; utilisé pour comprendre Playwright et les bonnes pratiques E2E.
- [11] Google for Developers, “Machine learning and web development educational videos.” YouTube videos, 2025. Consulté le 10/06/2026 ; utilisé pour les concepts ML et architecture web.
- [12] Hugging Face, “Sentence transformers and embeddings educational videos.” YouTube videos, 2025. Consulté le 15/06/2026 ; utilisé pour les embeddings et similarité sémantique.